# Chapter 5 ▪ Inheritance

**Chapter 4** introduced you to classes and objects. In this chapter, you will learn about *inheritance,* another fundamental concept of object-oriented programming. The idea behind inheritance is that you can create new classes that are built on existing classes. When you inherit from an existing class, you reuse (or inherit) its methods, and you can add new methods and fields to adapt your new class to new situations. This technique is essential in Java programming.

This chapter also covers *reflection*, the ability to find out more about classes and their properties in a running program. Reflection is a powerful feature, but it is undeniably complex. Since reflection is of greater interest to tool builders than to application programmers, you can probably glance over that part of the chapter upon first reading and come back to it later.

## 5.1. Classes, Superclasses, and Subclasses

Let's return to the `Employee` class discussed in the previous chapter. Suppose (alas) you work for a company where managers are treated differently from other employees. Managers are, of course, just like employees in many respects. Both employees and managers are paid a salary. However, while employees are expected to complete their assigned tasks in return for receiving their salary, managers get *bonuses* if they actually achieve what they are supposed to do. This is the kind of situation that cries out for inheritance. Why? Well, you need to define a new class, `Manager`, and add functionality. But you can retain some of what you have already programmed in the `Employee` class, and *all* the fields of the original class can be preserved. More abstractly, there is an obvious "is–a" relationship between `Manager` and `Employee`. Every manager *is an* employee: This "is–a" relationship is the hallmark of inheritance.

---



**Note:** In this chapter, I use the classic example of employees and managers, but I must ask you to take this example with a grain of salt. In the real world, an employee can become a manager, so you would want to model being a manager as a role of an employee, not a subclass. In this chapter, however, I assume the corporate world is populated by two kinds of people: those who are forever employees, and those who have always been managers.

More generally, inheritance may not be as common as was envisioned when object-oriented programming first became popular. Nevertheless, it is a powerful and indispensable notion in many programming situations.

### 5.1.1. Defining Subclasses

Here is how you define a `Manager` class that inherits from the `Employee` class. Use the Java keyword `extends` to denote inheritance.

```
public class Manager extends Employee {
    added methods and fields
}
```



**Note:** Some programming languages use a more cryptic syntax for inheritance than Java. In Python, you declare `class Manager(Employee)`, and in C++, you write `class Manager : public Employee`. All inheritance in Java is public inheritance; there is no analog to the C++ features of private and protected inheritance.

The keyword `extends` indicates that you are making a new class that derives from an existing class. The existing class is called the *superclass*, *base class*, or *parent class*. The new class is called the *subclass*, *derived class*, or *child class*. The terms superclass and subclass are those most commonly used by Java programmers, although some programmers prefer the parent/child analogy, which also ties in nicely with the "inheritance" theme.

The `Employee` class is a superclass, but not because it is superior to its subclass or contains more functionality. *In fact, the opposite is true:* Subclasses have *more* functionality than their superclasses. For example, as you will see when we go over the rest of the `Manager` class code, the `Manager` class encapsulates more data and has more functionality than its superclass `Employee`.



**Note:** The prefixes *super* and *sub* come from the language of sets used in theoretical computer science and mathematics. The set of all employees contains the set of all managers, and thus is said to be a *superset* of the set of managers. Or, to put it another way, the set of all managers is a *subset* of the set of all employees.

Our `Manager` class has a new field to store the bonus, and a new method to set it:

```
public class Manager extends Employee {
    private double bonus;
    . . .
    public void setBonus(double bonus) {
```

```
        this.bonus = bonus;
    }
}
```

There is nothing special about these methods and fields. If you have a `Manager` object, you can simply apply the `setBonus` method.

```
Manager boss = . . .;
boss.setBonus(5000);
```

Of course, if you have an `Employee` object, you cannot apply the `setBonus` method—it is not among the methods defined in the `Employee` class.

However, you *can* use methods such as `getName` and `getHireDay` with `Manager` objects. Even though these methods are not explicitly defined in the `Manager` class, they are automatically inherited from the `Employee` superclass.

Every `Manager` object has four fields: `name`, `salary`, `hireDay`, and `bonus`. The fields `name`, `salary`, and `hireDay` are taken from the superclass.



**Note:** The Java language specification states: "Members of a class that are declared private are not inherited by subclasses of that class." This has confused my readers over the years. The specification uses the word "inherits" narrowly. It considers the private fields non-inherited because the `Manager` class cannot access them directly. Thus, every `Manager` object has three fields from the superclass, but the `Manager` class does not "inherit" them.

When defining a subclass by extending its superclass, you only need to indicate the *differences* between the subclass and the superclass. When designing classes, you place the most general methods in the superclass and more specialized methods in its subclasses. Factoring out common functionality by moving it to a superclass is routine in object-oriented programming.



**Note:** In **Chapter 4**, you learned about *records*: classes whose state is entirely defined by the constructor parameters. You cannot extend a record, and a record cannot extend another class.

### 5.1.2. Overriding Methods

Some of the superclass methods are not appropriate for the `Manager` subclass. In particular, the `getSalary` method should return the sum of the base salary and the bonus. You need to supply a new method to *override* the superclass method:

```
public class Manager extends Employee {
    . . .
    public double getSalary() {
        . . .
    }
    . . .
}
```

How can you implement this method? At first glance, it appears to be simple—just return the sum of the `salary` and `bonus` fields:

```
public double getSalary() {
    return salary + bonus; // won't work
}
```

However, that won't work. Recall that only the `Employee` methods have direct access to the private fields of the `Employee` class. This means that the `getSalary` method of the `Manager` class cannot directly access the `salary` field. If the `Manager` methods want to access those private fields, they have to do what every other method does—use the public interface, in this case the public `getSalary` method of the `Employee` class.

So, let's try again. You need to call `getSalary` instead of simply accessing the `salary` field:

```
public double getSalary() {
    double baseSalary = getSalary(); // still won't work
    return baseSalary + bonus;
}
```

Now, the problem is that the call to `getSalary` simply calls *itself*, because the `Manager` class has a `getSalary` method (namely, the method we are trying to implement). The consequence is an infinite chain of calls to the same method, leading to a program crash.

We need to indicate that we want to call the `getSalary` method of the `Employee` superclass, not the current class. Use the special keyword `super` for this purpose. The call

```
super.getSalary()
```

calls the `getSalary` method of the `Employee` class. Here is the correct version of the `getSalary` method for the `Manager` class:

```
public double getSalary() {
    double baseSalary = super.getSalary();
    return baseSalary + bonus;
}
```



**Note:** Some people think of `super` as being analogous to the `this` reference. However, that analogy is not quite accurate: `super` is not a reference to an object. For example, you cannot assign the value `super` to another object variable. Instead, `super` is a special keyword that directs the compiler to invoke the superclass method.

As you saw, a subclass can *add* fields, and it can *add* methods or *override* the methods of the superclass. However, inheritance can never take away any fields or methods.

### 5.1.3. Subclass Constructors

To complete our example, let us supply a constructor.

```
public Manager(String name, double salary, int year, int month, int day) {
    super(name, salary, year, month, day);
    bonus = 0;
}
```

Here, the keyword `super` has a different meaning. The instruction

```
super(name, salary, year, month, day);
```

is shorthand for "call the constructor of the `Employee` superclass with `name`, `salary`, `year`, `month`, and `day` as arguments."

Since the `Manager` constructor cannot access the private fields of the `Employee` class, it must initialize them through a constructor. The constructor is invoked with the special `super` syntax.

Before Java 25, the call to `super` had to be the first statement in the constructor for the subclass. Now, the code between the start of the constructor and the call to `super` is an early execution context. As you have already seen in **Chapter 4**, only limited actions are permitted in this context. Except for assignment to uninitialized fields, you cannot access the object under construction in any way.

When a subclass object is constructed without an explicit invocation of a superclass constructor, the superclass must have a no-argument constructor. That constructor is invoked prior to the subclass construction.



**Note:** Recall that the `this` keyword has two meanings: to denote a reference to the implicit parameter and to call another constructor of the same class. Likewise, the `super` keyword has two meanings: to invoke a superclass method and to invoke a superclass constructor. When used to invoke constructors, the `this` and `super` keywords are closely related. The constructor parameters are either passed to another constructor of the same class (`this`) or a constructor of the superclass (`super`).

---

After you redefine the `getSalary` method for `Manager` objects, managers will *automatically* have the bonus added to their salaries.

Here's an example of this at work. We make a new manager and set the manager's bonus:

```
Manager boss = new Manager("Carl Cracker", 80000, 1987, 12, 15);
boss.setBonus(5000);
```

We make an array of three employees:

```
var staff = new Employee[3];
```

We populate the array with a mix of managers and employees:

```
staff[0] = boss;
staff[1] = new Employee("Harry Hacker", 50000, 1989, 10, 1);
staff[2] = new Employee("Tony Tester", 40000, 1990, 3, 15);
```

We print out everyone's salary:

```
for (Employee e : staff)
    IO.println(e.getName() + " " + e.getSalary());
```

This loop prints the following data:

```
Carl Cracker 85000.0
Harry Hacker 50000.0
Tommy Tester 40000.0
```

Now `staff[1]` and `staff[2]` each print their base salary because they are `Employee` objects. However, `staff[0]` is a `Manager` object whose `getSalary` method adds the bonus to the base salary.

What is remarkable is that the call

```
e.getSalary()
```

picks out the *correct* `getSalary` method. Note that the *declared* type of `e` is `Employee`, but the *actual* type of the object to which `e` refers can be either `Employee` or `Manager`.

When `e` refers to an `Employee` object, the call `e.getSalary()` calls the `getSalary` method of the `Employee` class. However, when `e` refers to a `Manager` object, then the `getSalary` method of the `Manager` class is called instead. The virtual machine knows about the actual type of the object to which `e` refers, and therefore can invoke the correct method.

The fact that an object variable (such as the variable `e`) can refer to multiple actual types is called *polymorphism*. Automatically selecting the appropriate method at runtime is called *dynamic binding*. I discuss both topics in more detail in this chapter.



**Note:** In C++ and C#, you need to declare a method as `virtual` if you want dynamic binding. In Java, dynamic binding is the default behavior; if you do *not* want a method to be virtual, you tag it as `final`. (I discuss the `final` keyword later in this chapter.)

Listing 5.1 contains a program that shows how the salary computation differs for `Employee` and `Manager` objects. The `Employee` class is the final version of the preceding chapter, and the `Manager` class is shown in Listing 5.2.

Starting with this chapter, classes are in packages and have (mostly) public methods, which is the form that you normally see them in production code. However, the class with the `main` method is kept as simple as possible.

You can launch the program in this section as

```
java InheritanceDemo.java
```

**Listing 5.1 v1ch05/InheritanceDemo.java**

```
1  package v1ch05;
2
3  import com.horstmann.corejava.*;
4
5  /**
6   * This program demonstrates inheritance.
7   */
8  class InheritanceDemo {
9      void main() {
10         // construct a Manager object
```

```
        var boss = new Manager("Carl Cracker", 80000, 1987, 12, 15);
        boss.setBonus(5000);

        var staff = new Employee[3];

        // fill the staff array with Manager and Employee objects

        staff[0] = boss;
        staff[1] = new Employee("Harry Hacker", 50000, 1989, 10, 1);
        staff[2] = new Employee("Tommy Tester", 40000, 1990, 3, 15);

        // print out information about all Employee objects
        for (Employee e : staff)
            IO.println("name=" + e.getName() + ",salary=" + e.getSalary());
    }
}
```

**Listing 5.2 com/horstmann/corejava/Manager.java**

```
package com.horstmann.corejava;

public class Manager extends Employee {
    private double bonus;

    public Manager(String name, double salary, int year, int month, int day) {
        super(name, salary, year, month, day);
        bonus = 0;
    }

    public double getSalary() {
        double baseSalary = super.getSalary();
        return baseSalary + bonus;
    }

    public void setBonus(double b) {
        bonus = b;
    }
}
```

### 5.1.4. Inheritance Hierarchies

Inheritance need not stop at deriving one layer of classes. We could have an `Executive` class that extends `Manager`, for example. The collection of all classes extending a common superclass is called an *inheritance hierarchy*, as shown in **Figure 5.1**. The path from a particular class to its ancestors in the inheritance hierarchy is its *inheritance chain*.

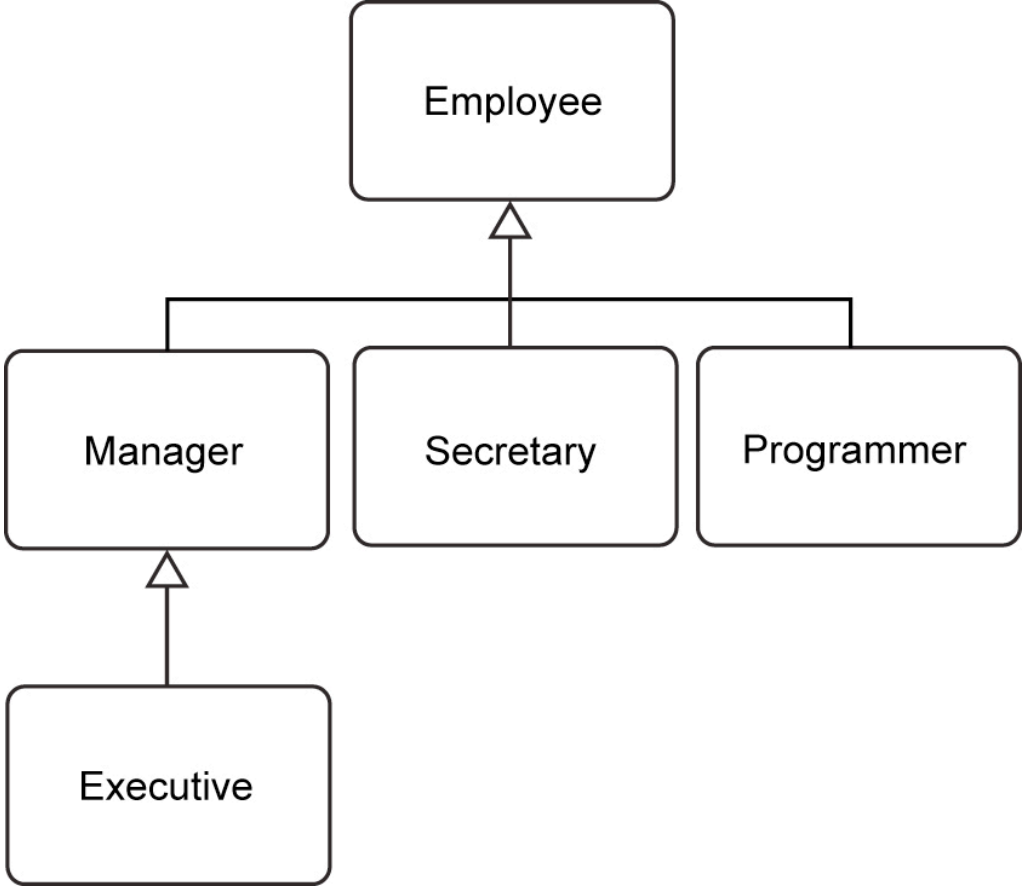



**Figure 5.1:** Employee inheritance hierarchy

There is usually more than one chain of descent from a distant ancestor class. You could form subclasses `Programmer` or `Secretary` that extend `Employee`, and they would have nothing to do with the `Manager` class (or with each other). This process can continue as long as is necessary.

---



**Note:** In Python and C++, a class can have multiple superclasses. Java does not support multiple inheritance. For ways to recover much of the functionality of multiple inheritance, see **Chapter 6**.

---

### 5.1.5. Polymorphism

A simple rule can help you decide whether or not inheritance is the right design for your data. The "is–a" rule states that every object of the subclass is an object of the superclass. For example, every manager is an employee. Thus, it makes sense for the `Manager` class to be a subclass of the `Employee` class. Naturally, the opposite is not true—not every employee is a manager.

Another way of formulating the "is–a" rule is the *substitution principle.* That principle states that you can use a subclass object whenever the program expects a superclass object.

For example, you can assign a subclass object to a superclass variable.

```
Employee e;
e = new Employee(. . .); // Employee object expected
e = new Manager(. . .); // OK, Manager can be used as well
```

In the Java programming language, object variables are *polymorphic*. A variable of type `Employee` can refer to an object of type `Employee` or to an object of any subclass of the `Employee` class (such as `Manager`, `Executive`, `Secretary`, and so on).

We took advantage of this principle in **Listing 5.1**:

```
Manager boss = new Manager(. . .);
Employee[] staff = new Employee[3];
staff[0] = boss;
```

In this case, the variables `staff[0]` and `boss` refer to the same object. However, `staff[0]` is considered to be only an `Employee` object by the compiler.

That means you can call

```
boss.setBonus(5000); // OK
```

but you can't call

```
staff[0].setBonus(5000); // ERROR
```

The declared type of `staff[0]` is `Employee`, and the `setBonus` method is not a method of the `Employee` class.

However, you cannot assign a superclass reference to a subclass variable. For example, it is not legal to make the assignment

```
Manager m = staff[i]; // ERROR
```

The reason is clear: Not all employees are managers. If this assignment were to succeed and `m` were to refer to an `Employee` object that is not a manager, then it would later be possible to call `m.setBonus(. . .)` and a runtime error would occur.

---



**Caution:** In Java, arrays of subclass references can be converted to arrays of superclass references without a cast. For example, consider this array of managers:

```
Manager[] managers = new Manager[10];
```

It is legal to convert this array to an `Employee[]` array:

```
Employee[] staff = managers; // OK
```

Sure, why not, you may think. After all, if `managers[i]` is a `Manager`, it is also an `Employee`. But actually, something surprising is going on. Keep in mind that `managers` and `staff` are references to the same array. Now consider the statement

```
staff[0] = new Employee("Harry Hacker", . . .);
```

The compiler will cheerfully allow this assignment. But `staff[0]` and `managers[0]` are the same reference, so it looks as if we managed to smuggle a mere employee into the management ranks. That would be very bad—calling `managers[0].setBonus(1000)` would try to access a nonexistent instance field and would corrupt neighboring memory.

To make sure no such corruption can occur, all arrays remember the element type with which they were created, and they monitor that only compatible references are stored into them. For example, the array created as `new Manager[10]` remembers that it is an array of managers. Attempting to store an `Employee` reference causes an `ArrayStoreException`.

---

### 5.1.6. Understanding Method Calls

It is important to understand exactly how a method call is applied to an object. Let's say we call `x.f(args)`, and the implicit argument `x` is declared to be an object of class `C`. Here is what happens:

1. The compiler looks at the declared type of the object and the method name. Note that there may be multiple methods, all with the same name, `f`, but with different parameter types. For example, there may be a method `f(int)` and a method `f(String)`. The compiler enumerates all methods called `f` in the class `C` and all accessible methods called `f` in the superclasses of `C`. (Private methods of the superclass are not accessible.)
   Now the compiler knows all possible candidates for the method to be called.
2. Next, the compiler determines the types of the arguments supplied in the method call. If among all the methods called `f` there is a unique method whose parameter types are a best match for the supplied arguments, that method is chosen to be called. This process is called *overloading resolution*. For example, in a call `x.f("Hello")`, the compiler picks `f(String)` and not `f(int)`. The situation can get complex because of type conversions (`int` to

`double`, `Manager` to `Employee`, and so on). If the compiler cannot find any method with matching parameter types or if multiple methods all match after applying conversions, the compiler reports an error.
Now the compiler knows the name and parameter types of the method that needs to be called.

---



**Note:** Recall that the name and parameter type list for a method are called the method's *signature*. For example, `f(int)` and `f(String)` are two methods with the same name but different signatures. If you define a method in a subclass that has the same signature as a superclass method, you override the superclass method.
The return type is not part of the signature. However, when you override a method, you need to keep the return type compatible. A subclass may change the return type to a subtype of the original type. For example, suppose the `Employee` class has a method

```
public Employee getBuddy() { . . . }
```

A manager would never want to have a lowly employee as a buddy. To reflect that fact, the `Manager` subclass can override this method as

```
public Manager getBuddy() { . . . } // OK to change return type
```

We say that the two `getBuddy` methods have *covariant* return types.

---

3. If the method is `private`, `static`, `final`, or a constructor, then the compiler knows exactly which method to call. (The `final` modifier is explained in the next section.) This is called *static binding*. Otherwise, the method to be called depends on the actual type of the implicit argument, and dynamic binding must be used at runtime. In our example, the compiler would generate an instruction to call `f(String)` with dynamic binding.

4. When the program runs and uses dynamic binding to call a method, the virtual machine must call the version of the method that is appropriate for the *actual* type of the object to which `x` refers. Let's say the actual type is `D`, a subclass of `C`. If the class `D` defines a method `f(String)`, that method is called. If not, `D`'s superclass is searched for a method `f(String)`, and so on.
It would be time-consuming to carry out this search every time a method is called. Instead, the virtual machine precomputes a *method table* for each class. The method table lists all method signatures and the actual methods to be called.
The virtual machine can build the method table after loading a class, by combining the methods that it finds in the class file with the method table of the superclass.
When a method is actually called, the virtual machine simply makes a table lookup. In our example, the virtual machine consults the method table for the class `D` and looks up the method to call for `f(String)`. That method may be

`D.f(String)` or `X.f(String)`, where `X` is some superclass of `D`. There is one twist to this scenario. If the call is `super.f(param)`, then the virtual machine consults the method table of the superclass.

Let's look at this process in detail in the call `e.getSalary()` in **Listing 5.1**. The declared type of `e` is `Employee`. The `Employee` class has a single method, called `getSalary`, with no method parameters. Therefore, in this case, we don't worry about overloading resolution.

The `getSalary` method is not `private`, `static`, or `final`, so it is dynamically bound. The virtual machine produces method tables for the `Employee` and `Manager` classes. The `Employee` table shows that all methods are defined in the `Employee` class itself:

```
Employee:
    getName() -> Employee.getName()
    getSalary() -> Employee.getSalary()
    getHireDay() -> Employee.getHireDay()
    raiseSalary(double) -> Employee.raiseSalary(double)
```

Actually, that isn't the whole story—as you will see later in this chapter, the `Employee` class has a superclass `Object` from which it inherits a number of methods. I ignore the `Object` methods for now.

The `Manager` method table is slightly different. Three methods are inherited, one method is redefined, and one method is added.

```
Manager:
    getName() -> Employee.getName()
    getSalary() -> Manager.getSalary()
    getHireDay() -> Employee.getHireDay()
    raiseSalary(double) -> Employee.raiseSalary(double)
    setBonus(double) -> Manager.setBonus(double)
```

At runtime, the call `e.getSalary()` is resolved as follows:

1. First, the virtual machine fetches the method table for the actual type of `e`. That may be the table for `Employee`, `Manager`, or another subclass of `Employee`.
2. Then, the virtual machine looks up the defining class for the `getSalary()` signature. Now it knows which method to call.
3. Finally, the virtual machine calls the method.

Dynamic binding has a very important property: It makes programs *extensible* without the need for modifying existing code. Suppose a new class `Executive` is added and there is the possibility that the variable `e` refers to an object of that

class. The code containing the call `e.getSalary()` need not be recompiled. The `Executive.getSalary()` method is called automatically if `e` happens to refer to an object of type `Executive`.



**Caution:** When you override a method, the subclass method must be *at least as visible* as the superclass method. In particular, if the superclass method is `public`, the subclass method must also be declared `public`. It is a common error to accidentally omit the `public` specifier for the subclass method. The compiler then complains that you try to supply a more restrictive access privilege.

### 5.1.7. Preventing Inheritance: Final Classes and Methods

Occasionally, you want to prevent someone from forming a subclass of one of your classes. Classes that cannot be extended are called *final* classes, and you use the `final` modifier in the definition of the class to indicate this. For example, suppose we want to prevent others from subclassing the `Executive` class. Simply declare the class using the `final` modifier, as follows:

```
public final class Executive extends Manager {
    . . .
}
```

You can also make a specific method in a class `final`. If you do this, then no subclass can override that method. (All methods in a `final` class are automatically `final`.) For example:

```
public class Employee {
    . . .
    public final String getName() {
        return name;
    }
    . . .
}
```



**Note:** Recall that fields can also be declared as `final`. A final field cannot be changed after the object has been constructed. However, if a class is declared `final`, only the methods, not the fields, are automatically `final`.

There is only one good reason to make a method or class `final`: to make sure its semantics cannot be changed in a subclass. For example, the `getTime` and `setTime` methods of the `Calendar` class are `final`. This indicates that the designers of the `Calendar` class have taken over responsibility for the conversion between the `Date` class and the calendar state. No subclass should be allowed to mess up this arrangement. Similarly, the `String` class is a `final` class. That

means nobody can define a subclass of `String`. In other words, if you have a `String` reference, you know it refers to a `String` and nothing but a `String`.

If you call a method in a constructor, you should declare it as `final`. Otherwise, it can be overridden in a subclass, and it can access a partially constructed subclass instance. Here is an example. For debugging purposes, the `Employee` constructor displays a description of the constructed object.

```
public class Employee {
    public Employee(String name, double salary, int year, int month, int day) {
        this.name = name;
        this.salary = salary;
        hireDay = LocalDate.of(year, month, day);
        IO.println("Constructed " + description());
    }

    public String description() {
        return "An employee with a salary of " + salary;
     }
     . . .
}
```

Now a new class is added to the hierarchy of employee classes—executives with titles:

```
public class Executive extends Manager {
    private String title;

    public Executive(String name, String title, double salary,
            int year, int month, int day) {
        super(name, salary, year, month, day);
        this.title = title;;
    }

    public String getTitle() {
        return title;
    }

    public String description() {
        if (title.length() >= 20)
            return "An executive with an impressive title";
        else
            return "An executive with a title of " + title;
    }
}
```

When an `Executive` is constructed, its constructor first calls the `Manager` constructor, which calls the `Employee` constructor, which calls the `description` method. Because of polymorphism, that is the `description` method in the

`Executive` class! Unfortunately, the `Executive` constructor hasn't finished yet. The `title` instance field is still `null`, causing a `NullPointerException`.

---



**Caution:** Calling a method in a constructor is inherently dangerous. The constructor must have done enough work for the method to function correctly. If the method can be overridden, this becomes very difficult to ensure. Therefore, it is best to call only `final` or `private` methods in a constructor.

Some languages handle this differently. In C++, method calls in a constructor are not polymorphic. For example, if you call `getDescription` in an `Employee` constructor, it always invokes `Employee::getDescription`.

---



**Tip:** Since Java 21, if you compile with the `-Xlint:this-escape` option, the compiler issues a warning when the constructor of a public class calls a method that is not `final` or `private`.

The name of the flag is a bit unfortunate since there are other situations where the `this` reference can "escape" from a constructor that the compiler does not currently detect.

---

Some programmers believe that you should declare all methods as `final` unless you have a good reason to want polymorphism. In fact, in C++ and C#, methods do not use polymorphism unless you specifically request it. That may be a bit extreme, but I agree that it is a good idea to think carefully about final methods and classes when you design a class hierarchy.

In the early days of Java, some programmers used the `final` keyword hoping to avoid the overhead of dynamic binding. If a method is not overridden, and it is short, then a compiler can optimize the method call away—a process called *inlining*. For example, inlining the call `e.getName()` replaces it with the field access `e.name`. This is a worthwhile improvement—CPUs hate branching because it interferes with their strategy of prefetching instructions while processing the current one. However, if `getName` can be overridden in another class, then the compiler cannot inline it because it has no way of knowing what the overriding code may do.

Fortunately, the just-in-time compiler in the virtual machine can do a better job than a traditional compiler. It knows exactly which classes extend a given class, and it can check whether any class actually overrides a given method. If a method is short, frequently called, and not actually overridden, the just-in-time compiler can inline it. What happens if the virtual machine loads another subclass that overrides an inlined method? Then the optimizer must undo the inlining. That takes time, but it happens rarely.

---



**Note:** Enumerations and records are always final—you cannot extend them.

---

### 5.1.8. Casting

Recall from **Chapter 3** that the process of forcing a conversion from one type to another is called casting. The Java programming language has a special notation for casts. For example,

```
double x = 3.405;
int nx = (int) x;
```

converts the value of the expression `x` into an integer, discarding the fractional part.

Just as you occasionally need to convert a floating-point number to an integer, you may need to convert an object reference from one class to another. Let's again use the example of an array containing a mix of `Employee` and `Manager` objects:

```
var staff = new Employee[3];
staff[0] = new Manager("Carl Cracker", 80000, 1987, 12, 15);
staff[1] = new Employee("Harry Hacker", 50000, 1989, 10, 1);
staff[2] = new Employee("Tony Tester", 40000, 1990, 3, 15);
```

To actually make a cast of an object reference, use a syntax similar to what you use for casting numeric expressions. Surround the target class name with parentheses and place it before the object reference you want to cast. For example:

```
Manager boss = (Manager) staff[0];
```

There is only one reason why you would want to make a cast—to use an object in its full capacity after its actual type has been temporarily forgotten. For example, in the `ManagerDemo` class, the `staff` array had to be an array of `Employee` objects because *some* of its elements were regular employees. We would need to cast the managerial elements of the array back to `Manager` to access any of its new variables. (Note that in the sample code for the first section, I made a special effort to avoid the cast. I initialized the `boss` variable with a `Manager` object before storing it in the array. I needed the correct type to set the bonus of the manager.)

As you know, in Java every variable has a type. The type describes the kind of object the variable refers to and what it can do. For example, `staff[i]` refers to an `Employee` object (so it can also refer to a `Manager` object).

The compiler checks that you do not promise too much when you store a value in a variable. If you assign a subclass reference to a superclass variable, you are promising less, and the compiler will simply let you do it. If you assign a super-

class reference to a subclass variable, you are promising more. Then you must use a cast so that your promise can be checked at runtime.

What happens if you try to cast down an inheritance chain and are "lying" about what an object contains?

```
Manager boss = (Manager) staff[1]; // ERROR
```

When the program runs, the Java runtime system notices the broken promise and generates a `ClassCastException`. If you do not catch the exception, your program terminates. Thus, it is good programming practice to find out whether a cast will succeed before attempting it. Simply use the `instanceof` operator. For example:

```
if (staff[i] instanceof Manager) {
    boss = (Manager) staff[i];
    . . .
}
```

Finally, the compiler will not let you make a cast if there is no chance for the cast to succeed. For example, the cast

```
String c = (String) staff[i];
```

is a compile-time error because `String` is not a subclass of `Employee`.

To sum up:

- You can cast only within an inheritance hierarchy.
- Use `instanceof` to check before casting from a superclass to a subclass.

---



**Note:** The test

```
x instanceof C
```

does not generate an exception if `x` is `null`. It simply returns `false`. That makes sense: `null` refers to no object, so it certainly doesn't refer to an object of type `C`.

---

Actually, converting the type of an object by a cast is not usually a good idea. In our example, you do not need to cast an `Employee` object to a `Manager` object for most purposes. The `getSalary` method will work correctly on both objects of both classes. The dynamic binding that makes polymorphism work locates the correct method automatically.

The only reason to make the cast is to use a method that is unique to managers, such as `setBonus`. If for some reason you find yourself wanting to call `setBonus` on `Employee` objects, ask yourself whether this is an indication of a design flaw in the superclass. It may make sense to redesign the superclass and add a `setBonus` method. Remember, it takes only one uncaught `ClassCastException` to terminate your program. In general, it is best to minimize the use of casts and the `instanceof` operator.

---



**Note:** Java uses the cast syntax from the "bad old days" of C, but it works like the safe `dynamic_cast` operation of C++. For example,

```
Employee[] staff = . . .;
Manager boss = (Manager) staff[i]; // Java
```

is equivalent to

```
Employee* staff[] = . . .;
Manager* boss = dynamic_cast<Manager*>(staff[i]); // C++
```

There is one notable difference. If the `dynamic_cast` fails, it yields a null pointer instead of throwing an exception.

---

### 5.1.9. Pattern Matching for `instanceof`

The code

```
if (staff[i] instanceof Manager) {
    Manager boss = (Manager) staff[i];
    boss.setBonus(5000);
}
```

is rather verbose. Do we really need to mention the subclass `Manager` three times?

As of Java 16, there is an easier way. You can declare the subclass variable right in the `instanceof` test:

```
if (staff[i] instanceof Manager boss) {
    boss.setBonus(5000);
}
```

If `staff[i]` is an instance of the `Manager` class, then the variable `boss` is set to `staff[i]`, and you can use it *as a* `Manager`. You skip the cast.

If `staff[i]` doesn't refer to a `Manager`, `boss` is not set, and the `instanceof` operator yields the value `false`. The body of the `if` statement is skipped.



**Tip:** In most situations in which you use `instanceof`, you need to apply a subclass method. Then use this "pattern-matching" form of `instanceof` instead of a cast.

A useless `instanceof` pattern is a compile-time error:

```
Manager boss = . . .;
if (boss instanceof Employee e) . . . // ERROR: Of course it's an Employee
```



**Note:** The equally useless

```
if (boss instanceof Employee) . . .
```

is allowed, for backward compatibility with Java 1.0.

When an `instanceof` pattern introduces a variable, you can use it right away, in the same expression:

```
Employee e = . . .;
if (e instanceof Executive exec && exec.getTitle().getLength() >= 20) . . .
```

This works because the right-hand side of an `&&` expression is only evaluated if the left-hand side is `true`. If the right-hand side is evaluated, `exec` must have been bound to an `Executive` instance.

However, the following is a compile-time error:

```
if (e instanceof Manager exec || exec.getTitle().getLength() >= 20) . . . // ERROR
```

The right-hand side of `||` is executed when the left-hand side is `false`, and then nothing is bound to the variable `exec`.

Here is another example with the conditional operator:

```
String title = e instanceof Executive exec ? exec.getTitle() : "";
```

The variable `exec` is defined in the subexpression after the `?`, but not in the subexpression after the `:`.



**Note:** The variable-declaring `instanceof` forms are called "pattern-matching" because they are similar to type patterns in `switch`, which are covered in detail in **Section 5.9**.



**Caution:** As any local variable, the local variable defined by a pattern can shadow a field. For example:

```
class Value {
    private double v;
    public boolean equals(Object other) {
        if (other instanceof LabeledValue v)
            // v is the same as other
        else
            // v denotes the field
    }
    . . .
}
```

The preceding example showed a *type pattern*. If the type matches, the object is bound to a variable of that type. When the type is a `record`, you can, as of Java 21, do better and declare variables that are bound to the components. This is called a *record pattern*:

```
record Point(double x, double y) {}
Point p = . . .;
if (p instanceof Point(var a, var b)) distance = Math.hypot(a, b);
```

Now `a` and `b` are bound to the `x` and `y` components of `p`. Of course, you could also invoke `p.x()` and `p.y()`, but a record pattern can be more concise.

For added clarity, you can also specify explicit types for the introduced variables:

```
if (p instanceof Point(double a, double b)) . . .;
```

Record patterns can be nested:

```
record Circle(Point center, double radius) {}
Circle c = . . .;
if (c instanceof Circle(Point(var a, var b), var r)) . . .;
```

Since Java 22, you can denote unmatched parts in a record pattern with an underscore:

```
if (p instanceof Point(var a, _)) distance = Math.abs(a);
```

### 5.1.10. Protected Access

As you know, fields in a class are best tagged as `private`, and methods are usually tagged as `public`. Any features declared `private` won't be accessible in other classes. As explained at the beginning of this chapter, this is also true for subclasses: A subclass cannot access the private fields of its superclass.

There are times, however, when you want to restrict a method to subclasses only or, less commonly, to allow subclass methods to access a superclass field. In that case, you declare a class feature as `protected`. For example, if the superclass `Employee` declares the `hireDay` field as `protected` instead of `private`, then the `Manager` methods can access it directly.

In Java, a protected field is accessible by any class in the same package. Now consider an `Administrator` subclass in a different package. The methods of the `Administrator` class can peek inside the `hireDay` field of `Administrator` objects only, not of other `Employee` objects. This restriction is made so that you can't abuse the `protected` mechanism by forming subclasses just to gain access to the protected fields.

In practice, use protected fields with caution. Suppose your class is used by other programmers and you designed it with protected fields. Unknown to you, other programmers may inherit classes from your class and start accessing your protected fields. In this case, you can no longer change the implementation of your class without upsetting those programmers. That is against the spirit of OOP, which encourages data encapsulation.

Protected methods make more sense. A class may declare a method as `protected` if it is tricky to use. This indicates that the subclasses (which, presumably, know their ancestor well) can be trusted to use the method correctly, but other classes cannot.

A good example of this kind of method is the `clone` method of the `Object` class —see **Chapter 6** for more details.

---



**Note:** As already mentioned, protected features in Java are accessible to all subclasses as well as to all other classes in the same package. This is slightly different from the C++ meaning of protected, and it makes the notion of `protected` in Java even less safe than in C++.

---

Here is a summary of the four access control modifiers in Java:

1. Accessible in the class only (`private`).
2. Accessible by the world (`public`).
3. Accessible in the package and all subclasses (`protected`).
4. Accessible in the package—the (unfortunate) default. No modifiers are needed.

## 5.2. `Object`: The Cosmic Superclass

The `Object` class is the ultimate ancestor—every class in Java extends `Object`. However, you never have to write

```
public class Employee extends Object
```

The ultimate superclass `Object` is taken for granted if no superclass is explicitly mentioned. Since *every* class in Java extends `Object`, it is important to be familiar with the services provided by the `Object` class. I go over the basic ones in this chapter; consult the later chapters or view the online documentation for what is not covered here. (Several methods of `Object` come up only when dealing with concurrency—see **Chapter 10**.)

### 5.2.1. Variables of Type `Object`

You can use a variable of type `Object` to refer to objects of any type:

```
Object obj = new Employee("Harry Hacker", 35000);
```

Of course, a variable of type `Object` is only useful as a generic holder for arbitrary values. To do anything specific with the value, you need to have some knowledge about the original type and apply a cast:

```
Employee e = (Employee) obj;
```

In Java, only the values of *primitive types* (numbers, characters, and `boolean` values) are not objects.

All array types, no matter whether they are arrays of objects or arrays of primitive types, are class types that extend the `Object` class.

```
Employee[] staff = new Employee[10];
obj = staff; // OK
obj = new int[10]; // OK
```



**Note:** In many object-oriented programming languages, all classes extend a common root class; for example, `object` in Python or `Object` in JavaScript. The exception is C++, where classes can be completely unrelated.

### 5.2.2. The `equals` Method

The `equals` method in the `Object` class tests whether one object is considered equal to another. The `equals` method, as implemented in the `Object` class, determines whether two object references are identical. This is a pretty reasonable default—if two objects are identical, they should certainly be equal. For quite a few classes, nothing else is required. For example, it makes little sense to compare two `PrintStream` objects for equality. However, you will often want to implement state-based equality testing, in which two objects are considered equal when they have the same state.

For example, let us consider two employees equal if they have the same name, salary, and hire date. (In an actual employee database, it would be more sensible to compare IDs instead. I use this example to demonstrate the mechanics of implementing the `equals` method.)

```
public class Employee {
    . . .
    public boolean equals(Object otherObject) {
        // a quick test to see if the objects are identical
        if (this == otherObject) return true;

        // must return false if the explicit parameter is null
        if (otherObject == null) return false;

        // if the classes don't match, they can't be equal
        if (getClass() != otherObject.getClass())
           return false;

        // now we know otherObject is a non-null Employee
        Employee other = (Employee) otherObject;

        // test whether the fields have identical values
        return name.equals(other.name)
           && salary == other.salary
           && hireDay.equals(other.hireDay);
    }
}
```

The `getClass` method returns the class of an object—we discuss this method in detail later in this chapter. In our test, two objects can only be equal when they belong to the same class.

---



**Tip:** To guard against the possibility that `name` or `hireDay` are `null`, use the `Objects.equals` method. The value of `Objects.equals(a, b)` is `true` if both arguments are `null`, `false` if only one is `null`, and `a.equals(b)` otherwise. With that method, the last statement of the `Employee.equals` method becomes

```
 return Objects.equals(name, other.name)
     && salary == other.salary
     && Objects.equals(hireDay, other.hireDay);
```



**Caution:** When a class has few fields, the test `if (this == otherObject)` may block the virtual machine from inlining calls to `equals`. Consider benchmarking both versions.

When you define the `equals` method for a subclass, first call `equals` on the superclass. If that test doesn't pass, then the objects can't be equal. If the superclass fields are equal, you are ready to compare the instance fields of the subclass.

```
public class Manager extends Employee {
    . . .
    public boolean equals(Object otherObject) {
        if (!super.equals(otherObject)) return false;
        // super.equals checked that this and otherObject belong to the same class
        Manager other = (Manager) otherObject;
        return bonus == other.bonus;
    }
}
```



**Note:** Recall from **Chapter 4** that a record is a special form of an immutable class whose state is entirely defined by the fields set in a "canonical" constructor. Records automatically define an `equals` method that compares the components. Two record instances are equal when the corresponding component values are equal.



**Caution:** Components of type `double` are compared with `Double.equals`, not with `==`. This makes a difference for `-0.0` and `Double.NaN`.

### 5.2.3. Equality Testing and Inheritance

How should the `equals` method behave if the implicit and explicit parameters don't belong to the same class? This has been an area of some controversy. In the preceding example, the `equals` method returns `false` if the classes don't match exactly. But many programmers use an `instanceof` test instead:

```
if (!(otherObject instanceof Employee)) return false;
```

This leaves open the possibility that `otherObject` can belong to a subclass. However, this approach can get you into trouble. Here is why. The Java Language Specification requires that the `equals` method has the following properties:

1. It is *reflexive*: For any non-`null` reference `x`, `x.equals(x)` should return `true`.
2. It is *symmetric*: For any references `x` and `y`, `x.equals(y)` should return `true` if and only if `y.equals(x)` returns `true`.
3. It is *transitive*: For any references `x`, `y`, and `z`, if `x.equals(y)` returns `true` and `y.equals(z)` returns `true`, then `x.equals(z)` should return `true`.
4. It is *consistent*: If the objects to which `x` and `y` refer haven't changed, then repeated calls to `x.equals(y)` return the same value.
5. For any non-`null` reference `x`, `x.equals(null)` should return `false`.

These rules are certainly reasonable. You wouldn't want a library implementor to ponder whether to call `x.equals(y)` or `y.equals(x)` when locating an element in a data structure.

However, the symmetry rule has subtle consequences when the parameters belong to different classes. Consider a call

```
e.equals(m)
```

where `e` is an `Employee` object and `m` is a `Manager` object, both of which happen to have the same name, salary, and hire date. If `Employee.equals` uses an `instanceof` test, the call returns `true`. But that means that the reverse call

```
m.equals(e)
```

also needs to return `true`—the symmetry rule does not allow it to return `false` or to throw an exception.

That leaves the `Manager` class in a bind. Its `equals` method must be willing to compare itself to any `Employee`, without taking manager-specific information into account! All of a sudden, the `instanceof` test looks less attractive.

Some authors have gone on record that the `getClass` test is wrong because it violates the substitution principle. A commonly cited example is the `equals` method in the `AbstractSet` class that tests whether two sets have the same elements. The `AbstractSet` class has two concrete subclasses, `TreeSet` and `HashSet`, that use different algorithms for locating set elements. You really want to be able to compare any two sets, no matter how they are implemented.

However, the set example is rather specialized. It would make sense to declare `AbstractSet.equals` as `final`, because nobody should redefine the semantics of set equality. (The method is not actually `final`. This allows a subclass to implement a more efficient algorithm for the equality test.)

The way I see it, there are two distinct scenarios:

- If subclasses can have their own notion of equality, then the symmetry requirement forces you to use the `getClass` test.
- If the notion of equality is fixed in the superclass, then you can use the `instanceof` test and allow objects of different subclasses to be equal to one another.

In the example with employees and managers, we consider two objects to be equal when they have matching fields. If we have two `Manager` objects with the same name, salary, and hire date, but with different bonuses, we want them to be different. Therefore, we use the `getClass` test.

But suppose we used an employee ID for equality testing. This notion of equality makes sense for all subclasses. Then we could use the `instanceof` test, and we should have declared `Employee.equals` as `final`.

---



**Note:** The standard Java library contains over 150 implementations of `equals` methods, with a mishmash of using `instanceof`, calling `getClass`, catching a `ClassCastException`, or doing nothing at all. Check out the API documentation of the `java.sql.Timestamp` class, where the implementors note with some embarrassment that they have painted themselves in a corner. The `Timestamp` class inherits from `java.util.Date`, whose `equals` method uses an `instanceof` test, and it is impossible to override `equals` to be both symmetric and accurate.

---



**Caution:** The commonly cited example of set equality is a cautionary tale, showing just how difficult it is to provide symmetric equality across subclasses.

Consider these two sets:

```
Set<String> x = new HashSet<>();
x.add("Hello");
x.add("World");
Set<String> y = new TreeSet<>(String.CASE_INSENSITIVE_ORDER);
y.add("Hello");
y.add("WORLD");
```

With these sets, `equals` is not symmetric:

```
x.equals(y) // false
y.equals(x) // true
```

`TreeSet` uses the sort order, and not `equals`, to compare elements. With case-insensitive order, the strings `"World"` and `"WORLD"` are deemed

equal.

---

Here is a recipe for writing the perfect `equals` method:

1. Name the explicit parameter `otherObject`—later, you will need to cast it to another variable that you should call `other`.
2. Test whether `this` happens to be identical to `otherObject`:

   ```
   if (this == otherObject) return true;
   ```

   This statement is an optimization that can worthwhile for classes with complex state. It may be counterproductive in some cases, though.
3. Test whether `otherObject` is `null` and return `false` if it is. This test is required.

   ```
   if (otherObject == null) return false;
   ```

4. Compare the classes of `this` and `otherObject`. If the semantics of `equals` can change in subclasses, use the `getClass` test:

   ```
   if (getClass() != otherObject.getClass()) return false;
   ClassName other = (ClassName) otherObject;
   ```

   If the same semantics holds for *all* subclasses, you can use an `instanceof` test instead:

   ```
   if (!(otherObject instanceof ClassName other)) return false;
   ```

   Note that the `instanceof` test sets `other` to `otherObject` if it succeeds. No cast is necessary.
5. Now compare the fields, as required by your notion of equality. Use `==` for primitive type fields, `Objects.equals` for object fields. Return `true` if all fields match, `false` otherwise.

   ```
   return field1 == other.field1
       && Objects.equals(field2, other.field2)
       && . . .;
   ```

   If you redefine `equals` in a subclass, include a call to `super.equals(other)`.

---



**Tip:** If you have fields of array type, you can use the static `Arrays.equals` method to check that the corresponding array elements are equal. Use the

`Arrays.deepEquals` method for multidimensional arrays.



**Caution:** Here is a common mistake when implementing the `equals` method. Can you spot the problem?

```
public class Employee {
    public boolean equals(Employee other) {
        return other != null
            && getClass() == other.getClass()
            && Objects.equals(name, other.name)
            && salary == other.salary
            && Objects.equals(hireDay, other.hireDay);
    }
    . . .
}
```

This method declares the explicit parameter type as `Employee`. As a result, it does not override the `equals` method of the `Object` class but defines a completely unrelated method.

You can protect yourself against this type of error by tagging methods that are intended to override superclass methods with `@Override`:

```
@Override public boolean equals(Object other)
```

If you made a mistake and are defining a new method, the compiler reports an error. For example, suppose you add the following declaration to the `Employee` class:

```
@Override public boolean equals(Employee other)
```

An error is reported because this method doesn't override any method from the `Object` superclass.

`java.util.Arrays` **1.2**

- `static boolean equals(xxx[] a, xxx[] b)` **5.0**
  returns `true` if the arrays have equal lengths and equal elements in corresponding positions. The component type xxx of the array can be `Object`, `int`, `long`, `short`, `char`, `byte`, `boolean`, `float`, or `double`.

`java.util.Objects` 7

- `static boolean equals(Object a, Object b)` returns `true` if `a` and `b` are both `null`, `false` if exactly one of them is `null`, and `a.equals(b)` otherwise.

### 5.2.4. The `hashCode` Method

A hash code is an integer that is derived from an object. Hash codes should be scrambled—if `x` and `y` are two distinct objects, there should be a high probability that `x.hashCode()` and `y.hashCode()` are different. **Table 5.1** lists a few examples of hash codes that result from the `hashCode` method of the `String` class.

**Table 5.1:** Hash Codes Resulting from the `hashCode` Method

| String | Hash Code |
|---|---|
| `Hello` | `69609650` |
| `Harry` | `69496448` |
| `Hacker` | `-2141031506` |

The `String` class uses the following algorithm to compute the hash code:

```
int hash = 0;
for (int i = 0; i < length(); i++)
    hash = 31 * hash + charAt(i);
```

The `hashCode` method is defined in the `Object` class. Therefore, every object has a default hash code, called the *identity hash code*. How that hash code is determined depends on the virtual machine. Consider this example:

```
var s = "Ok";
var sb = new StringBuilder(s);
IO.println(s.hashCode() + " " + sb.hashCode());
var t = new String("Ok");
var tb = new StringBuilder(t);
IO.println(t.hashCode() + " " + tb.hashCode());
```

**Table 5.2** shows the result.

**Table 5.2:** Hash Codes of Strings and String Builders

| Object | Hash Code | Object | Hash Code |
|---|---|---|---|

| Object | Hash Code | Object | Hash Code |
|---|---|---|---|
| `s` | `2556` | `t` | `2556` |
| `sb` | `20526976` | `tb` | `20527144` |

Note that the strings `s` and `t` have the same hash code because, for strings, the hash codes are derived from their *contents*. The string builders `sb` and `tb` have different hash codes because no `hashCode` method has been defined for the `StringBuilder` class, and the default `hashCode` method in the `Object` class derives the hash code from the object's memory address.

If you redefine the `equals` method, you will also need to redefine the `hashCode` method for objects that users might insert into a hash table. (I discuss hash tables in **Chapter 9**.)

The `hashCode` method should return an integer (which can be negative). Just combine the hash codes of the instance fields so that the hash codes for different objects are likely to be widely scattered.

For example, here is a `hashCode` method for the `Employee` class:

```
public class Employee {
    public int hashCode() {
        return 7 * name.hashCode()
            + 11 * Double.valueOf(salary).hashCode()
            + 13 * hireDay.hashCode();
    }
    . . .
}
```

However, you can do better. First, use the null-safe method `Objects.hashCode`. It returns `0` if its argument is `null` and the result of calling `hashCode` on the argument otherwise. Also, use the static `Double.hashCode` method to avoid creating a `Double` object:

```
public int hashCode() {
    return 7 * Objects.hashCode(name)
        + 11 * Double.hashCode(salary)
        + 13 * Objects.hashCode(hireDay);
}
```

---



**Caution:** If the instance fields have small ranges of possible values, you need to achieve as many distinct hash codes as possible. Consider hashing calendar dates. Computing `7 * year + 11 * month + 13 * day` yields many collisions. In contrast, `31 * 12 * year + 31 * month + day` is a

"perfect hash function." Assuming a reasonable year range, no two dates have the same hash code. (The actual `hashCode` method of the `LocalDate` class, which supports a range of ±999,999,999 years, is a bit more complex.)

---

Combining multiple hash values can be tedious. In many cases, you can simply call `Objects.hash` with all fields. It will combine the hash codes of its arguments. Then the `Employee.hashCode` method is simply

```
public int hashCode() {
    return Objects.hash(name, salary, hireDay);
}
```

However, you need to be careful with fields that are arrays. The hash code of an array is computed with `Object.hashCode` and does *not*depend on its elements. Instead, use the static `Arrays.hashCode` method which combines the hash codes of the array elements. There are overloads for primitive type arrays and arrays of objects. For multi-dimensional arrays, use `Arrays.deepHashCode`.

---



**Note:** The `Arrays.hashCode(Object[])` method works almost like `String.hashCode`, using the "multiply with 31 and add" operation to combine element hash codes. But there is a twist. A zero-length array has hash code 1, to distinguish it from `null`. Here is the implementation:

```
public static int hashCode(Object[] a) {
    if (a == null) return 0;

    int result = 1; // Starting at 1, not 0

    for (Object element : a)
        result = 31 * result + (element == null ? 0 : element.hashCode());

    return result;
}
```

The `Objects.hash` method calls `Arrays.hashCode`. As a consequence, `Objects.hash(x)` equals `31 + x.hashCode()`.

---

A `record` type automatically provides a `hashCode` method that derives a hash code from the hash codes of the component values. Its exact behavior is left open in the Java Language Specification. In OpenJDK, the component hashes are simply combined with the "multiply with 31 and add" operation, starting with zero.

Your definitions of `equals` and `hashCode` must be compatible: If `x.equals(y)` is `true`, then `x.hashCode()` must return the same value as `y.hashCode()`. For example, if you define `Employee.equals` to compare employee IDs, then the

`hashCode` method needs to hash the IDs, not employee names or memory addresses.

`java.lang.Object` 1.0

- `int hashCode()`
  returns a hash code for this object. A hash code can be any integer, positive or negative. Equal objects need to return identical hash codes.

`java.util.Objects` 7

- `static int hash(Object... objects)`
  returns a hash code that is combined from the hash codes of all supplied objects.
- `static int hashCode(Object a)`
  returns `0` if `a` is `null` or `a.hashCode()` otherwise.

`java.lang.Integer` 1.0

`java.lang.Long` 1.0

`java.lang.Short` 1.0

`java.lang.Byte` 1.0

`java.lang.Double` 1.0

`java.lang.Float` 1.0

`java.lang.Character` 1.0

`java.lang.Boolean` 1.0

- `static int hashCode(xxx value)` 8
  returns the hash code of the given value. Here `xxx` is the primitive type corresponding to the given wrapper type. (See for wrapper types.)

`java.util.Arrays` 1.2

- `static int hashCode(xxx[] a)` 5.0
  computes the hash code of the array `a`. The component type xxx of the array can be `Object`, `int`, `long`, `short`, `char`, `byte`, `boolean`, `float`, or `double`.

### 5.2.5. The `toString` Method

Another important method in `Object` is the `toString` method that returns a string representing the value of this object. Here is a typical example. The `toString` method of the `Point` class returns a string like this:

```
java.awt.Point[x=10,y=20]
```

Most (but not all) `toString` methods follow this format: the name of the class, then the field values enclosed in square brackets. Here is an implementation of the `toString` method for the `Employee` class:

```
public String toString() {
    return "Employee[name=" + name
        + ",salary=" + salary
        + ",hireDay=" + hireDay
        + "]";
}
```

Actually, you can do a little better. Instead of hardwiring the class name into the `toString` method, call `getClass().getName()` to obtain a string with the class name.

```
public String toString() {
    return getClass().getName()
        + "[name=" + name
        + ",salary=" + salary
        + ",hireDay=" + hireDay
        + "]";
}
```

Such `toString` method will also work for subclasses.

Of course, the subclass programmer should define its own `toString` method and add the subclass fields. If the superclass uses `getClass().getName()`, then the subclass can simply call `super.toString()`. For example, here is a `toString` method for the `Manager` class:

```
public class Manager extends Employee {
    . . .
    public String toString() {
        return super.toString()
            + "[bonus=" + bonus
            + "]";
    }
}
```

Now a `Manager` object is printed as

```
Manager[name=. . .,salary=. . .,hireDay=. . .][bonus=. . .]
```

The `toString` method is ubiquitous for an important reason: Whenever an object is concatenated with a string by the "+" operator, the Java compiler automatically invokes the `toString` method to obtain a string representation of the object. For example:

```
var p = new Point(10, 20);
String message = "The current position is " + p;
    // automatically invokes p.toString()
```

---



**Tip:** Instead of writing `x.toString()`, you can write `"" + x`. This statement concatenates the empty string with the string representation of `x` that is exactly `x.toString()`. Unlike `toString`, this statement even works if `x` is of primitive type.

---

If `x` is any object and you call

```
IO.println(x);
```

then the `println` method simply calls `x.toString()` and prints the resulting string.

The `Object` class defines the `toString` method to print the class name and the hash code of the object. For example, the call

```
IO.println(System.err)
```

produces an output that looks like this:

```
java.io.PrintStream@2f6684
```

The reason is that the implementor of the `PrintStream` class didn't bother to override the `toString` method.

---



**Caution:** Annoyingly, arrays inherit the `toString` method from `Object`, with the added twist that the array type is printed in an archaic format. For example,

```
int[] luckyNumbers = { 2, 3, 5, 7, 11, 13 };
```

```
String s = "" + luckyNumbers;
```

yields the string `"[I@1a46e30"`. (The prefix `[I` denotes an array of integers.) The remedy is to call the static `Arrays.toString` method instead. The code

```
String s = Arrays.toString(luckyNumbers);
```

yields the string `"[2, 3, 5, 7, 11, 13]"`.

To correctly print multidimensional arrays (that is, arrays of arrays), use `Arrays.deepToString`.

---

The `toString` method is a great tool for logging. Many classes in the standard class library define the `toString` method so that you can get useful information about the state of an object. This is particularly useful in logging messages like this:

```
IO.println("Current position = " + position);
```

As explained in **Chapter 7**, an even better solution is to use a logger object and call

```
logger.log(INFO, "Current position = " + position);
```

---



**Tip:** I strongly recommend that you add a `toString` method to each class that you write. You, as well as other programmers who use your classes, will be grateful for the logging support.

However, for `record` types, a `toString` method is already provided. It simply lists the class name and the names and stringified values of the fields.

---



**Note:** In the preceding sections, you learned how to override the `equals`, `hashCode`, and `toString` methods. In the perhaps unlikely case that you need access to the unmodified behavior, do this:

- Compare with `==` instead of `equals`
- Call `System.indentityHashCode(obj)` or `Objects.identityToString(obj)` to get what would have been returned if you had not overriden `hashCode` or `toString`

---

The program in **Listing 5.3** tests the `equals`, `hashCode`, and `toString` methods for the classes `Employee` (**Listing 5.4**) and `Manager` (**Listing 5.5**).

**Listing 5.3 v1ch05/equals/EqualsDemo.java**

```
package v1ch05.equals;

/**
 * This program demonstrates the equals method.
 */
class EqualsDemo {
    void main() {
        var alice1 = new Employee("Alice Adams", 75000, 1987, 12, 15);
        var alice2 = alice1;
        var alice3 = new Employee("Alice Adams", 75000, 1987, 12, 15);
        var bob = new Employee("Bob Brandson", 50000, 1989, 10, 1);

        IO.println("alice1 == alice2: " + (alice1 == alice2));

        IO.println("alice1 == alice3: " + (alice1 == alice3));

        IO.println("alice1.equals(alice3): " + alice1.equals(alice3));

        IO.println("alice1.equals(bob): " + alice1.equals(bob));

        IO.println("bob.toString(): " + bob);

        var carl = new Manager("Carl Cracker", 80000, 1987, 12, 15);
        var boss = new Manager("Carl Cracker", 80000, 1987, 12, 15);
        boss.setBonus(5000);
        IO.println("boss.toString(): " + boss);
        IO.println("carl.equals(boss): " + carl.equals(boss));
        IO.println("alice1.hashCode(): " + alice1.hashCode());
        IO.println("alice3.hashCode(): " + alice3.hashCode());
        IO.println("bob.hashCode(): " + bob.hashCode());
        IO.println("carl.hashCode(): " + carl.hashCode());
    }
}
```

**Listing 5.4 v1ch05/equals/Employee.java**

```
package v1ch05.equals;

import module java.base;

/**
 * The familiar Employee class, with equals, hashCode, and toString methods.
 */
public class Employee {
    private String name;
    private double salary;
    private LocalDate hireDay;

    public Employee(String name, double salary, int year, int month, int day) {
        this.name = name;
```

```
        this.salary = salary;
        hireDay = LocalDate.of(year, month, day);
    }

    public String getName() {
        return name;
    }

    public double getSalary() {
        return salary;
    }

    public LocalDate getHireDay() {
        return hireDay;
    }

    public void raiseSalary(double byPercent) {
        double raise = salary * byPercent / 100;
        salary += raise;
    }

    public boolean equals(Object otherObject) {
        // a quick test to see if the objects are identical
        if (this == otherObject) return true;

        // must return false if the explicit parameter is null
        if (otherObject == null) return false;

        // if the classes don't match, they can't be equal
        if (getClass() != otherObject.getClass()) return false;

        // now we know otherObject is a non-null Employee
        var other = (Employee) otherObject;

        // test whether the fields have identical values
        return Objects.equals(name, other.name) && salary == other.salary
                && Objects.equals(hireDay, other.hireDay);
    }

    public int hashCode() {
        return Objects.hash(name, salary, hireDay);
    }

    public String toString() {
        return getClass().getName() + "[name=" + name + ",salary=" + salary
                + ",hireDay=" + hireDay + "]";
    }
}
```

**Listing 5.5 v1ch05/equals/Manager.java**

```
1  package v1ch05.equals;
2
3  /**
4   * The familiar Manager class, with equals, hashCode, and toString methods.
5   */
6  public class Manager extends Employee {
7      private double bonus;
8
9      public Manager(String name, double salary, int year, int month, int day) {
10         super(name, salary, year, month, day);
11         bonus = 0;
12     }
13
14     public double getSalary() {
15         double baseSalary = super.getSalary();
16         return baseSalary + bonus;
17     }
18
19     public void setBonus(double bonus) {
20         this.bonus = bonus;
21     }
22
23     public boolean equals(Object otherObject) {
24         if (!super.equals(otherObject)) return false;
25         var other = (Manager) otherObject;
26         // super.equals checked that this and other belong to the same class
27         return bonus == other.bonus;
28     }
29
30     public int hashCode() {
31         return java.util.Objects.hash(super.hashCode(), bonus);
32     }
33
34     public String toString() {
35         return super.toString() + "[bonus=" + bonus + "]";
36     }
37 }
```

`java.lang.Object` 1.0

- `Class getClass()`
  returns a class object that contains information about the object. As you will see later in this chapter, Java has a runtime representation for classes that is encapsulated in the `Class` class.
- `boolean equals(Object otherObject)`
  compares two objects for equality; returns `true` if the objects point to the same area of memory, and `false` otherwise. You should override this method in your own classes.

- `String toString()`
  returns a string that represents the value of this object. You should override this method in your own classes.

`java.lang.Class` 1.0

- `String getName()`
  returns the name of this class.
- `Class getSuperclass()`
  returns the superclass of this class as a `Class` object.

`java.lang.System` 1.0

- `static int identityHashCode(Object x)` 1.1
  returns the hash code that `Object.hashCode` would return for `x` if not overridden, or 0 if `x` is `null`.

`java.util.Objects` 7

- `static String identityToString(Object o)` 19
  returns the string that `Object.toString` would return for `o` if not overridden. Throws a `NullPointerException` if `o` is `null`.

## 5.3. Generic Array Lists

In some programming languages—in particular, in C and C++—you have to fix the sizes of all arrays at compile time. Programmers hate this because it forces them into uncomfortable tradeoffs. How many employees will be in a department? Surely no more than 100. What if there is a humongous department with 150 employees? Do we want to waste 90 entries for every department with just 10 employees?

In Java, the situation is somewhat better. You can set the size of an array at runtime.

```
int actualSize = . . .;
var staff = new Employee[actualSize];
```

Of course, this code does not completely solve the problem of dynamically modifying arrays at runtime. Once you set the array size, you cannot change it easily. Instead, in Java you can deal with this common situation by using another Java class, called `ArrayList`. The `ArrayList` class is similar to an array, but it automatically adjusts its capacity as you add and remove elements, without any additional code.

`ArrayList` is a *generic class* with a *type parameter*. To specify the type of the element objects that the array list holds, you append a class name enclosed in angle brackets, such as `ArrayList<Employee>`. You will see in **Chapter 8** how to define your own generic class, but you don't need to know any of those technicalities to use the `ArrayList` type.

The following sections show you how to work with array lists.

### 5.3.1. Declaring Array Lists

Here is how to declare and construct an array list that holds `Employee` objects:

```
ArrayList<Employee> staff = new ArrayList<Employee>();
```

It is a good idea to use the `var` keyword to avoid duplicating the class name:

```
var staff = new ArrayList<Employee>();
```

It you don't use the `var` keyword, you can omit the type argument on the right-hand side:

```
ArrayList<Employee> staff = new ArrayList<>();
```

This is called the "diamond" syntax because the empty brackets `<>` resemble a diamond. Use the diamond syntax together with the `new` operator. The compiler checks what happens to the new value. If it is assigned to a variable, passed into a method, or returned from a method, then the compiler checks the generic type of the variable, parameter, or method. It then places that type into the `<>`. In our example, the `new ArrayList<>()` is assigned to a variable of type `ArrayList<Employee>`. Therefore, the generic type is `Employee`.



**Caution:** If you declare an `ArrayList` with `var`, do *not* use the diamond syntax. The declaration

```
var elements = new ArrayList<>();
```

yields an `ArrayList<Object>`.



**Note:** Before Java 5, there were no generic classes. Instead, there was a single `ArrayList` class, a one-size-fits-all collection holding elements of type `Object`. You can still use `ArrayList` without a `<. . .>` suffix. It is considered a "raw" type, with the type parameter erased.



**Note:** In even older versions of Java, programmers used the `Vector` class for dynamic arrays. However, the `ArrayList` class is more efficient, and there is no longer any good reason to use the `Vector` class.

---

Use the `add` method to add new elements to an array list. For example, here is how you populate an array list with `Employee` objects:

```
staff.add(new Employee("Harry Hacker", . . .));
staff.add(new Employee("Tony Tester", . . .));
```

The array list manages an internal array of object references. Eventually, that array will run out of space. This is where array lists work their magic: If you call `add` and the internal array is full, the array list automatically creates a bigger array and copies all the objects from the smaller to the bigger array.

If you already know, or have a good guess, how many elements you want to store, call the `ensureCapacity` method before filling the array list:

```
staff.ensureCapacity(100);
```

That call allocates an internal array of 100 objects. Then, the first 100 calls to `add` will not involve any costly reallocation.

You can also pass an initial capacity to the `ArrayList` constructor:

```
ArrayList<Employee> staff = new ArrayList<>(100);
```

---



**Caution:** Allocating an array list as

```
new ArrayList<>(100) // capacity is 100
```

is *not* the same as allocating a new array as

```
new Employee[100] // size is 100
```

There is an important distinction between the capacity of an array list and the size of an array. If you allocate an array with 100 entries, then the array has 100 slots, ready for use. An array list with a capacity of 100 elements has the *potential* of holding 100 elements (and, in fact, more than 100, at the cost of additional reallocations)—but at the beginning, even after its initial construction, an array list holds no elements at all.

---

The `size` method returns the actual number of elements in the array list. For example,

```
staff.size()
```

returns the current number of elements in the `staff` array list. This is the equivalent of

```
a.length
```

for an array `a`.

Once you are reasonably sure that the array list is at its permanent size, you can call the `trimToSize` method. This method adjusts the size of the memory block to use exactly as much storage space as is required to hold the current number of elements. The garbage collector will reclaim any excess memory.

Once you trim the size of an array list, adding new elements will move the block again, which takes time. You should only use `trimToSize` when you are sure you won't add any more elements to the array list.



**Note:** The `ArrayList` class is similar to the C++ `vector` template. Both `ArrayList` and `vector` are generic types. But the C++ `vector` template overloads the `[]` operator for convenient element access. Java does not have operator overloading, so it must use explicit method calls instead. Moreover, C++ vectors are copied by value. If `a` and `b` are two vectors, then the assignment `a = b` makes `a` into a new vector with the same length as `b`, and all elements are copied from `b` to `a`. The same assignment in Java makes both `a` and `b` refer to the same array list.

`java.util.ArrayList<E>` **1.2**

- `ArrayList<E>()`
  constructs an empty array list.
- `ArrayList<E>(int initialCapacity)`
  constructs an empty array list with the specified capacity.
- `boolean add(E obj)`
  appends `obj` at the end of the array list. Always returns `true`.
- `int size()`
  returns the number of elements currently stored in the array list. (Of course, this is never larger than the array list's capacity.)
- `void ensureCapacity(int capacity)`
  ensures that the array list has the capacity to store the given number of elements without reallocating its internal storage array.

- `void trimToSize()`
  reduces the storage capacity of the array list to its current size.

### 5.3.2. Accessing Array List Elements

Unfortunately, nothing comes for free. The automatic growth convenience of array lists requires a more complicated syntax for accessing the elements. The reason is that the `ArrayList` class is not a part of the Java programming language; it is just a utility class programmed by someone and supplied in the standard library.

Instead of the pleasant `[]` syntax to access or change the element of an array, you use the `get` and `set` methods.

For example, to set the `i` th element, use

```
staff.set(i, harry);
```

This is equivalent to

```
a[i] = harry;
```

for an array `a`. (As with arrays, the index values are zero-based.)

---



**Caution:** Do not call `list.set(i, x)` until the *size* of the array list is larger than `i`. For example, the following code is wrong:

```
var list = new ArrayList<Employee>(100); // capacity 100, size 0
list.set(0, x); // no element 0 yet
```

Use the `add` method instead of `set` to fill up an array, and use `set` only to replace a previously added element.

---

To get an array list element, use

```
Employee e = staff.get(i);
```

This is equivalent to

```
Employee e = a[i];
```

---



**Note:** When there were no generic classes, the `get` method of the raw `ArrayList` class had no choice but to return an `Object`. Consequently,

callers of `get` had to cast the returned value to the desired type:

```
Employee e = (Employee) staff.get(i);
```

The raw `ArrayList` is also a bit dangerous. Its `add` and `set` methods accept objects of any type. A call

```
staff.set(i, "Harry Hacker");
```

compiles without so much as a warning, and you run into grief only when you retrieve the object and try to cast it. If you use an `ArrayList<Employee>` instead, the compiler will detect this error.

---

You can sometimes get the best of both worlds—flexible growth and convenient element access—with the following trick. First, make an array list and add all the elements:

```
var list = new ArrayList<X>();
while (. . .) {
    x = . . .;
    list.add(x);
}
```

When you are done, use the `toArray` method to copy the elements into an array:

```
var a = new X[list.size()];
list.toArray(a);
```

Sometimes, you need to add elements in the middle of an array list. Use the `add` method with an index argument:

```
int n = staff.size() / 2;
staff.add(n, e);
```

The elements at locations `n` and above are shifted up to make room for the new entry. If the new size of the array list after the insertion exceeds the capacity, the array list reallocates its storage array.

Similarly, you can remove an element from the middle of an array list:

```
Employee e = staff.remove(n);
```

The elements located above it are copied down, and the size of the array is reduced by one.

Inserting and removing elements is not terribly efficient. It is probably not worth worrying about for small array lists. But if you store many elements and frequently insert and remove in the middle of a collection, consider using a linked list instead. I explain how to program with linked lists in **Chapter 9**.

You can use the "for each" loop to traverse the contents of an array list:

```
for (Employee e : staff)
    do something with e
```

This loop has the same effect as

```
for (int i = 0; i < staff.size(); i++) {
    Employee e = staff.get(i);
    do something with e
}
```

**Listing 5.6** is a modification of the `EmployeeDemo` program of **Chapter 4**. The `Employee[]` array is replaced by an `ArrayList<Employee>`. Note the following changes:

- You don't have to specify the array size.
- You use `add` to add as many elements as you like.
- You use `size()` instead of `length` to count the number of elements.
- You use `a.get(i)` instead of `a[i]` to access an element.

**Listing 5.6 v1ch05/ArrayListDemo.java**

```
1  package v1ch05;
2
3  import module java.base;
4  import com.horstmann.corejava.*;
5
6  /**
7   * This program demonstrates the ArrayList class.
8   */
9  class ArrayListDemo {
10     void main() {
11         // fill the staff array list with three Employee objects
12         var staff = new ArrayList<Employee>();
13
14         staff.add(new Employee("Carl Cracker", 75000, 1987, 12, 15));
15         staff.add(new Employee("Harry Hacker", 50000, 1989, 10, 1));
16         staff.add(new Employee("Tony Tester", 40000, 1990, 3, 15));
17
```

```
        // raise everyone's salary by 5%
        for (Employee e : staff)
            e.raiseSalary(5);

        // print out information about all Employee objects
        for (Employee e : staff)
            IO.println("name=" + e.getName() + ",salary=" + e.getSalary()
                    + ",hireDay=" + e.getHireDay());
    }
}
```

`java.util.ArrayList<E>` 1.2

- `E set(int index, E obj)`
  puts the value `obj` in the array list at the specified index, returning the previous contents.
- `E get(int index)`
  gets the value stored at a specified index.
- `void add(int index, E obj)`
  shifts up elements to insert `obj` at the specified index.
- `E remove(int index)`
  removes the element at the given index and shifts down all elements above it. The removed element is returned.

### 5.3.3. Compatibility between Typed and Raw Array Lists

In your own code, you will always want to use type parameters for added safety. In this section, you will see how to interoperate with legacy code that does not use type parameters.

Suppose you have the following legacy class:

```
public class EmployeeDB {
    public void update(ArrayList list) { . . . }
    public ArrayList find(String query) { . . . }
}
```

You can pass a typed array list to the `update` method without any casts.

```
ArrayList<Employee> staff = . . .;
employeeDB.update(staff);
```

The `staff` object is simply passed to the `update` method.

---



**Caution:** Even though you get no error or warning from the compiler, this call is not completely safe. The `update` method might add elements into

the array list that are not of type `Employee`. When these elements are retrieved, an exception occurs. This sounds scary, but if you think about it, the behavior is simply as it was before generics were added to Java. The integrity of the virtual machine is never jeopardized. In this situation, you do not lose security, but you also do not benefit from the compile-time checks.

---

Conversely, when you assign a raw `ArrayList` to a typed one, you get a warning.

```
ArrayList<Employee> result = employeeDB.find(query); // yields warning
```

---



**Note:** To see the text of the warning, compile with the option `-Xlint:unchecked`.

---

Using a cast does not make the warning go away.

```
ArrayList<Employee> result = (ArrayList<Employee>) employeeDB.find(query);
    // yields another warning
```

Instead, you get a different warning, telling you that the cast is misleading.

This is the consequence of a somewhat unfortunate limitation of generic types in Java. For compatibility, the compiler translates all typed array lists into raw `ArrayList` objects after checking that the type rules were not violated. In a running program, all array lists are the same—there are no type parameters in the virtual machine. Thus, the casts `(ArrayList)` and `(ArrayList<Employee>)` carry out identical runtime checks.

There isn't much you can do about that situation. When you interact with legacy code, study the compiler warnings and satisfy yourself that the warnings are not serious.

Once you are satisfied, you can tag the variable that receives the cast with the `@SuppressWarnings("unchecked")` annotation, like this:

```
@SuppressWarnings("unchecked") ArrayList<Employee> result
    = (ArrayList<Employee>) employeeDB.find(query); // yields another warning
```

## 5.4. Object Wrappers and Autoboxing

Occasionally, you need to convert a primitive type like `int` to an object. All primitive types have class counterparts. For example, a class `Integer` corresponds to the primitive type `int`. These kinds of classes are usually called *wrappers*. The wrapper classes have obvious names: `Integer`, `Long`, `Float`, `Double`, `Short`, `Byte`, `Character`, and `Boolean`. (The first six inherit from the common superclass `Number`.) The wrapper classes are immutable—you cannot

change a wrapped value after the wrapper has been constructed. They are also `final`, so you cannot subclass them.

Suppose we want an array list of integers. Unfortunately, the type argument inside the angle brackets cannot be a primitive type. It is not possible to form an `ArrayList<int>`. Here, the `Integer` wrapper class comes in. It is OK to declare an array list of `Integer` objects.

```
var list = new ArrayList<Integer>();
```



**Caution:** An `ArrayList<Integer>` is far less efficient than an `int[]` array because each value is separately wrapped inside an object. You would only want to use this construct for small collections when programmer convenience is more important than efficiency.

Fortunately, there is a useful feature that makes it easy to add an element of type `int` to an `ArrayList<Integer>`. The call

```
list.add(3);
```

is automatically translated to

```
list.add(Integer.valueOf(3));
```

This conversion is called *autoboxing*.



**Note:** You might think that *autowrapping* would be more consistent, but the "boxing" metaphor was taken from C#.

Conversely, when you assign an `Integer` object to an `int` value, it is automatically unboxed. That is, the compiler translates

```
int n = list.get(i);
```

into

```
int n = list.get(i).intValue();
```

Automatic boxing and unboxing also takes place in arithmetic expressions. For example, you can apply the increment operator to a wrapper reference:

```
Integer n = 3;
```

```
n++;
```

The compiler automatically inserts instructions to unbox the object, increment the resulting value, and box it back.

In most cases, you get the illusion that the primitive types and their wrappers are one and the same. There is just one point in which they differ considerably: identity. As you know, the `==` operator, applied to wrapper objects, only tests whether the objects have identical memory locations. The following comparison would therefore probably fail:

```
Integer a = 1000;
Integer b = 1000;
if (a == b) . . .
```

However, a Java implementation *may*, if it chooses, wrap commonly occurring values into identical objects, and thus the comparison might succeed. This ambiguity is not what you want. The remedy is to call the `equals` method when comparing wrapper objects.



**Note:** The autoboxing specification requires that `boolean`, `byte`, `char` <= `127`, `short`, and `int` between `-128` and `127` are wrapped into fixed objects. For example, if `a` and `b` had been initialized with `100` in the preceding example, then the comparison would have had to succeed.



**Tip:** Never rely on the identity of wrapper objects. Don't compare them with `==` and don't use them as locks (see **Chapter 10**).

Don't use the wrapper class constructors. They are deprecated and scheduled for removal. For example, use `Integer.valueOf(1000)`, never `new Integer(1000)`. Or, simply rely on autoboxing: `Integer a = 1000`.



**Note:** For the special values `-0.0` and `Double.NaN`, the `equals` method of the `Double` class is more convenient than the `==` operator. While `0.0 == -0.0`, the result of `Double.valueOf(-0.0).equals(Double.valueOf(0.0))` is `false`. Moreover, `Double.NaN != Double.NaN`, but `Double.valueOf(Double.NaN).equals(Double.valueOf(Double.NaN))` is `true`.

Mathematically speaking, `equals` is an "equivalence relation", but `==` is not.

There are a couple of other subtleties about boxing and unboxing. First off, since wrapper class references can be `null`, it is possible for unboxing to throw a `NullPointerException`:

```
Integer n = null;
IO.println(2 * n); // throws NullPointerException
```

Also, if you mix `Integer` and `Double` types in a conditional expression, then the `Integer` value is unboxed, promoted to `double`, and boxed into a `Double`:

```
Integer n = 1;
Double x = 2.0;
IO.println(true ? n : x); // prints 1.0
```

Finally, keep in mind that *compiler*, not the virtual machine, is in charge of boxing and unboxing. The compiler inserts the necessary calls when it generates the bytecodes of a class. The virtual machine simply executes those bytecodes.

---



**Preview:** A future version of Java will allow *value types*, user-defined types that are like primitive types—with values that are not stored inside objects. For example, a value of a primitive type `Point` with `double` fields `x` and `y` is simply a 16-byte block in memory, with two adjacent `double` values. You can copy it, but you can't have a reference to it.

It is planned that at some point, the primitive wrapper classes will become value classes.

---

You will often see the number wrappers for another reason. The designers of Java found the wrappers a convenient place to put certain basic methods, such as those for converting strings of digits to numbers.

To convert a string to an integer, use the following statement:

```
int x = Integer.parseInt(s);
```

This has nothing to do with `Integer` objects—`parseInt` is a static method. But the `Integer` class was a good place to put it.

The API notes show some of the more important methods of the `Integer` class. The other number classes implement corresponding methods.

The wrapper classes also define constants. For all integral wrapper types, `MIN_VALUE` and `MAX_VALUE` are the smallest and largest value of the type. For example, `Integer.MIN_VALUE` is `-2147483648`, and `Integer.MAX_VALUE` is `2147483647`. Here is a typical example:

```
int largest = Integer.MIN_VALUE; // Replaced with first input
while (in.hasNextInt()) largest = Math.max(largest, in.nextInt());
```



**Caution:** The `MIN_VALUE` constant of the `Double` and `Float` classes is the smallest *positive* value. If you want to compute the largest floating-point number and there may be negative inputs, you need to use:

```
int largest = -Double.MAX_VALUE; // Do NOT use Double.MIN_VALUE
while (in.hasNextDouble()) largest = Math.max(largest, in.nextDouble());
```



**Caution:** Some people think that the wrapper classes can be used to implement methods that can modify numeric parameters. However, that is not correct. Recall from **Chapter 4** that it is impossible to write a Java method that increments an integer parameter because parameters to Java methods are always passed by value.

```
public static void triple(int x) { // won't work
    x = 3 * x; // modifies local variable
}
```

Could we overcome this by using an `Integer` instead of an int?

```
public static void triple(Integer x) { // won't work
    . . .
}
```

The problem is that `Integer` objects are *immutable*: The information contained inside the wrapper can't change. You cannot use these wrapper classes to create a method that modifies numeric parameters.

`java.lang.Integer` 1.0

- `int intValue()`
  returns the value of this `Integer` object as an `int` (overrides the `intValue` method in the `Number` class).
- `static String toString(int i)`
  returns a new `String` object representing the number `i` in base 10.
- `static String toString(int i, int radix)`
  lets you return a representation of the number `i` in the base specified by the `radix` parameter.
- `static int parseInt(String s)`
- `static int parseInt(String s, int radix)`
  return the integer whose digits are contained in the string `s`. The string must represent an integer in base 10 (for the first method) or in the base given by the `radix` parameter (for the second method).

- `static Integer valueOf(String s)`
- `static Integer valueOf(String s, int radix)`
  return a new `Integer` object initialized to the integer whose digits are contained in the string `s`. The string must represent an integer in base 10 (for the first method) or in the base given by the `radix` parameter (for the second method).

`java.text.NumberFormat` 1.1

- `Number parse(String s)`
  returns the numeric value, assuming the specified `String` represents a number.

## 5.5. Methods with a Variable Number of Arguments

It is possible to provide methods that can be called with a variable number of arguments. These are sometimes called "varargs" methods.

You have already seen such a method: the `formatted` method of the `String` class. For example, the calls

```
"%d".formatted(n)
```

and

```
"%d %s".formatted(n, "widgets")
```

both call the same method, even though one call has two arguments and the other has three.

The `formatted` method is defined like this:

```
public class String {
    . . .
    public String formatted(Object... args) {
        . . .
    }
}
```

Here, the ellipsis `...` is part of the Java code. It denotes that the method can receive an arbitrary number of objects (in addition to the `fmt` parameter).

The `formatted` method has a parameter of type `Object[]` that holds all values to be formatted. (If the caller supplies integers or other primitive type values, autoboxing turns them into objects.) It now faces the unenviable task of scanning

the format string and matching up the `i`th format specifier with the value `args[i]`.

In other words, for the implementor of `formatted`, the `Object...` parameter type is exactly the same as `Object[]`.

The compiler needs to transform each call to `formatted`, bundling the arguments into an array and autoboxing as necessary:

```
"%d %s".formatted(new Object[] { Integer.valueOf(n), "widgets" } )
```

You can define your own methods with variable arguments, and you can specify any type for the arguments, even a primitive type. Here is a simple example: a function that computes the maximum of a variable number of values.

```
public static double max(double... values) {
    double largest = Double.NEGATIVE_INFINITY;
    for (double v : values) if (v > largest) largest = v;
    return largest;
}
```

Simply call the function like this:

```
double m = max(3.1, 40.4, -5);
```

The compiler passes a `new double[] { 3.1, 40.4, -5 }` to the `max` function.



**Note:** It is legal to pass an array of the variable argument type instead of the variable arguments. For example, you can call:

```
"%d %s".formatted(new Object[] { Integer.valueOf(1), "widgets" } )
```

Therefore, you can redefine an existing method whose last argument is an array to a method with variable arguments, without breaking any existing code. If you like, you can even declare the `main` method as

```
public static void main(String... args)
```

## 5.6. Abstract Classes

As you move up the inheritance hierarchy, classes become more general and probably more abstract. At some point, the ancestor class becomes *so* general that

you think of it more as a basis for other classes than as a class with specific instances you want to use. Consider, for example, an extension of our `Employee` class hierarchy. An employee is a person, and so is a student. Let us extend our class hierarchy to include classes `Person` and `Student`. **Figure 5.2** shows the inheritance relationships between these classes.

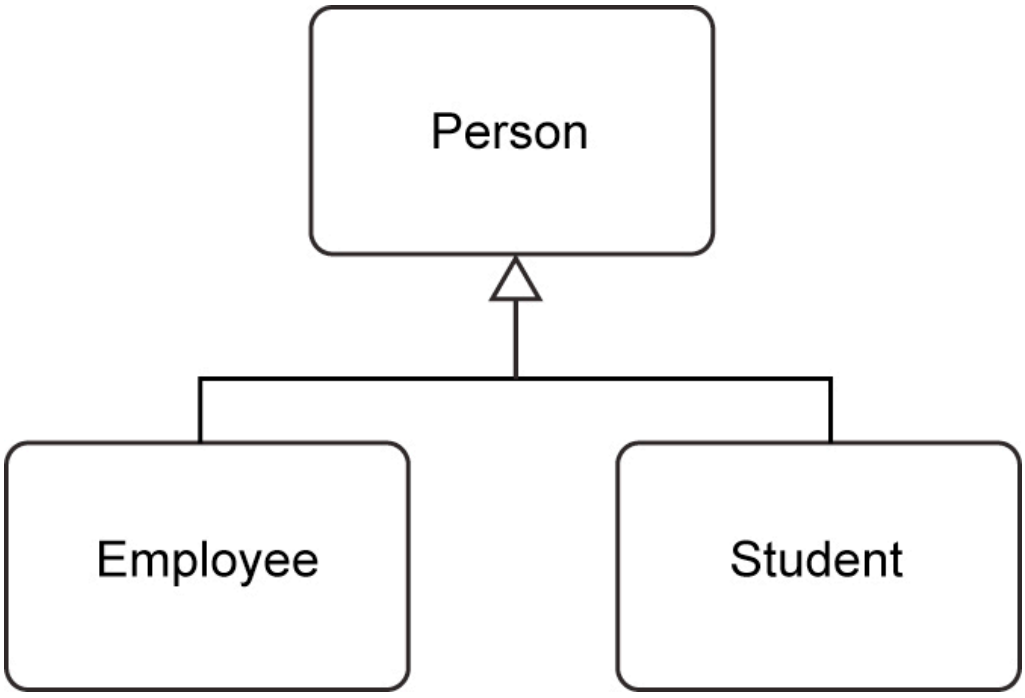



**Figure 5.2:** Inheritance diagram for `Person` and its subclasses

Why bother with so high a level of abstraction? There are some attributes that make sense for every person, such as a name. Both students and employees have names, and introducing a common superclass lets us factor out the `getName` method to a higher level in the inheritance hierarchy.

Now let's add another method, `getDescription`, whose purpose is to return a brief description of the person, such as

```
an employee with a salary of $50,000.00
a student majoring in computer science
```

It is easy to implement this method for the `Employee` and `Student` classes. But what information can you provide in the `Person` class? The `Person` class knows nothing about the person except the name. Of course, you could implement `Person.getDescription()` to return an empty string. But there is a better way. If you use the `abstract` keyword, you do not need to implement the method at all.

```
public abstract String getDescription();
    // no implementation required
```

For added clarity, a class with one or more abstract methods must itself be declared abstract.

```
public abstract class Person {
    . . .
    public abstract String getDescription();
}
```

In addition to abstract methods, abstract classes can have fields and concrete methods. For example, the `Person` class stores the name of the person and has a concrete method that returns it.

```
public abstract class Person {
    private String name;

    public Person(String name) {
        this.name = name;
    }

    public abstract String getDescription();

    public String getName() {
        return name;
    }
}
```

---



**Tip:** Some programmers don't realize that abstract classes can have concrete methods. You should always move common fields and methods (whether abstract or not) to the superclass (whether abstract or not).

---

Abstract methods act as placeholders for methods that are implemented in the subclasses. When you extend an abstract class, you have two choices. You can leave some or all of the abstract methods undefined; then, you must tag the subclass as abstract as well. Or, you can define all methods, and the subclass is no longer abstract.

For example, we will define a `Student` class that extends the abstract `Person` class and implements the `getDescription` method. None of the methods of the `Student` class are abstract, so it does not need to be declared as an abstract class.

A class can even be declared as `abstract` though it has no abstract methods.

Abstract classes cannot be instantiated. That is, if a class is declared as `abstract`, no objects of that class can be created. For example, the expression

```
new Person("Vince Vu")
```

is an error. However, you can create objects of concrete subclasses.

Note that you can still create *object variables* of an abstract class, but such a variable must refer to an object of a nonabstract subclass. For example:

```
Person p = new Student("Vince Vu", "Economics");
```

Here `p` is a variable of the abstract type `Person` that refers to an instance of the nonabstract subclass `Student`.

---



**Note:** In Python, abstract methods have an empty body and an `@abstractmethod` decorator.

In C++, an abstract method is called a *pure virtual function* and is tagged with a trailing `= 0`.

---

Let us define a concrete subclass `Student` that extends the abstract class `Person`:

```
public class Student extends Person {
    private String major;

    public Student(String name, String major) {
        super(name);
        this.major = major;
    }

    public String getDescription() {
        return "a student majoring in " + major;
    }
}
```

The `Student` class defines the `getDescription` method. Therefore, all methods in the `Student` class are concrete, and the class is no longer an abstract class.

The program shown in **Listing 5.7** defines the abstract superclass `Person` (**Listing 5.8**) and two concrete subclasses, `Employee` (**Listing 5.9**) and `Student` (**Listing 5.10**). We fill an array of `Person` references with employee and student objects:

```
var people = new Person[2];
people[0] = new Employee(. . .);
people[1] = new Student(. . .);
```

We then print the names and descriptions of these objects:

```
for (Person p : people)
    IO.println(p.getName() + ", " + p.getDescription());
```

Some people are baffled by the call

```
p.getDescription()
```

Isn't this a call to an undefined method? Keep in mind that the variable `p` never refers to a `Person` object because it is impossible to construct an object of the abstract `Person` class. The variable `p` always refers to an object of a concrete subclass such as `Employee` or `Student`. For these objects, the `getDescription` method is defined.

Could you have omitted the abstract method altogether from the `Person` superclass, simply defining the `getDescription` methods in the `Employee` and `Student` subclasses? If you did that, you wouldn't have been able to invoke the `getDescription` method on the variable `p`. The compiler ensures that you invoke only methods that are declared in the class.

Abstract methods are an important concept in the Java programming language. You will encounter them most commonly inside *interfaces*. For more information about interfaces, turn to **Chapter 6**.

**Listing 5.7 v1ch05/abstractClasses/AbstractClassesDemo.java**

```
1  package v1ch05.abstractClasses;
2
3  /**
4   * This program demonstrates abstract classes.
5   */
6  class AbstractClassesDemo {
7      void main() {
8          var people = new Person[2];
9
10         // fill the people array with Student and Employee objects
11         people[0] = new Employee("Harry Hacker", 50000, 1989, 10, 1);
12         people[1] = new Student("Maria Morris", "computer science");
13
14         // print out names and descriptions of all Person objects
15         for (Person p : people)
16             IO.println(p.getName() + ", " + p.getDescription());
17     }
18 }
```

**Listing 5.8 v1ch05/abstractClasses/Person.java**

```
package v1ch05.abstractClasses;

public abstract class Person {
    public abstract String getDescription();

    private String name;

    public Person(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }
}
```

**Listing 5.9 v1ch05/abstractClasses/Employee.java**

```
package v1ch05.abstractClasses;

import module java.base;

/**
 * The familiar Employee class, modified to extend the abstract class Person.
 */
public class Employee extends Person {
    private double salary;
    private LocalDate hireDay;

    public Employee(String name, double salary, int year, int month, int day) {
        super(name);
        this.salary = salary;
        hireDay = LocalDate.of(year, month, day);
    }

    public double getSalary() {
        return salary;
    }

    public LocalDate getHireDay() {
        return hireDay;
    }

    public String getDescription() {
        return "an employee with a salary of $%.2f".formatted(salary);
    }

    public void raiseSalary(double byPercent) {
        double raise = salary * byPercent / 100;
```

```
32          salary += raise;
33      }
34  }
```

**Listing 5.10 v1ch05/abstractClasses/Student.java**

```
1  package v1ch05.abstractClasses;
2
3  public class Student extends Person {
4      private String major;
5
6      /**
7       * @param name the student's name
8       * @param major the student's major
9       */
10     public Student(String name, String major) {
11         // pass name to superclass constructor
12         super(name);
13         this.major = major;
14     }
15
16     public String getDescription() {
17         return "a student majoring in " + major;
18     }
19 }
```

## 5.7. Enumeration Classes

You saw in **Chapter 3** how to define enumerated types. Here is a typical example:

```
public enum Size { SMALL, MEDIUM, LARGE, EXTRA_LARGE }
```

When you refer to these constants, qualify them with the class name, such as `Size.SMALL`. You can use the simple name in three situations:

1. As you will see shortly, enumerations can have methods. Inside the methods of an enumeration, you can use the simple name.
2. In a `case` of a `switch` whose selector has an enumerated type, you don't need the qualification:

   ```
   Size s = . . .;
   String abbrev = switch (s) {
       case SMALL -> "S";
       . . .
   };
   ```

3. You can statically import all constants of an enumeration:

```
import com.horstmann.util.Size;
import static com.horstmann.util.Size.*;
. . .
Size s = SMALL;
```



**Caution:** You cannot statically import constants from an enumeration in the default package. For example, if `Size` is in the default package, you cannot use

```
import static Size.*;
```

The type defined by an `enum` declaration is actually a class. The class has exactly four instances—it is not possible to construct new objects.

Therefore, you never need to use `equals` for values of enumerated types. Simply use `==` to compare them.



**Note:** You are allowed to put a comma after the last value. Trailing commas can be useful if you expect the number of values to grow.

```
public enum Size {
    SMALL,
    MEDIUM,
    LARGE,
    EXTRA_LARGE,
    XXL,
    // add more values here and put a comma after each
}
```

All enumerated types are subclasses of the abstract class `Enum`. They inherit a number of methods from that class. The most useful one is `name`, which returns the name of the enumerated constant. For example, `Size.SMALL.toString()` returns the string `"SMALL"`.

By default, the `toString` method returns `name`. However, you can override `toString`, whereas `name` is `final`.

The converse of `name` is the static `valueOf` method. For example, the statement

```
Size s = Enum.valueOf(Size.class, "SMALL");
```

sets `s` to `Size.SMALL`.

Each enumerated type has a static `values` method that returns an array of all values of the enumeration. For example, the call

```
Size[] values = Size.values();
```

returns the array with elements `Size.SMALL`, `Size.MEDIUM`, `Size.LARGE`, and `Size.EXTRA_LARGE`.

The `ordinal` method yields the position of an enumerated constant in the `enum` declaration, counting from zero. For example, `Size.MEDIUM.ordinal()` returns `1`.

You can, if you like, add constructors, methods, and fields to an enumerated type. Of course, the constructors are only invoked when the instances are constructed. Here is an example:

```
public enum Size {
    SMALL("S"), MEDIUM("M"), LARGE("L"), EXTRA_LARGE("XL");

    private final String abbreviation;

    Size(String abbreviation) { this.abbreviation = abbreviation; }
        // automatically private

    public String getAbbreviation() { return abbreviation; }
}
```

---



**Note:** When an `enum` has fields or methods, you must terminate the list of constants with a semicolon, as in the example above.

It is legal to have a trailing comma and then a semicolon.

---

You can even override methods in each instance:

```
public enum Size {
    SMALL,
    MEDIUM,
    LARGE,
    EXTRA_LARGE {
        public String toString() { return "XL"; }
    };
}
```

However, this is not common. In the preceding example, `EXTRA_LARGE` is an instance of a subclass of `Size`. Apart from this, you cannot form subclasses of an enumeration.

The constructor of an enumeration is always private. You can omit the `private` modifier, as in the preceding example. It is a syntax error to declare an enum constructor as `public` or `protected`.



**Note:** An enumeration with no instances cannot be instantiated. This is useful for indicating that a class is a utility class, with only static methods:

```
public enum FileUtils {
    ; // No instances

    public static String extension(String filename) {
        int n = filename.lastIndexOf(".");
        return n >= 0 ? filename.substring(n + 1) : "";
    }
    . . .
}
```



**Caution:** The *constructor* of an enumeration class cannot invoke a `switch` on the instance that is being constructed:

```
public enum Size {
    SMALL, MEDIUM, LARGE, EXTRA_LARGE;
    private final boolean largish;
    Size() {
        largish = switch (this) { // ERROR
            case LARGE, EXTRA_LARGE -> true;
            default -> false;
        };
    }
}
```

This curious limitation is caused by the implementation strategy for `switch` on `enum`. Each `switch` consults an array that maps the ordinal values of the `enum` constants, as currently defined, to the ordinal values of the time when the `switch` was compiled. These arrays can only be built after the `enum` instances are constructed.

In a method, such a `switch` works fine.

The short program in **Listing 5.11** demonstrates how to work with enumerated types.



**Note:** The `Enum` class has a type parameter that I have ignored for simplicity. For example, the enumerated type `Size` actually extends `Enum<Size>`. The type parameter is used in the `compareTo` method. (I discuss the `compareTo` method in **Chapter 6** and type parameters in **Chapter 8**.)

**Listing 5.11 v1ch05/EnumDemo.java**

```
package v1ch05;

/**
 * This program demonstrates enumerated types.
 */
class EnumDemo {
    void main() {
        String input = IO
                .readln("Enter a size: (SMALL, MEDIUM, LARGE, EXTRA_LARGE): ")
                .toUpperCase();
        Size size = Enum.valueOf(Size.class, input);
        IO.println("size=" + size);
        IO.println("abbreviation=" + size.getAbbreviation());
        if (size == Size.EXTRA_LARGE)
            IO.println("Good job--you paid attention to the _.");
    }
}

enum Size {
    SMALL("S"), MEDIUM("M"), LARGE("L"), EXTRA_LARGE("XL");

    private Size(String abbreviation) {
        this.abbreviation = abbreviation;
    }

    public String getAbbreviation() {
        return abbreviation;
    }

    private String abbreviation;
}
```

`java.lang.Enum<E>` **5.0**

- `static Enum valueOf(Class enumClass, String name)`
  returns the enumerated constant of the given class with the given name.
- `String name()`
  returns the name of this enumerated constant.
- `int ordinal()`
  returns the zero-based position of this enumerated constant in the `enum` declaration.
- `int compareTo(E other)`
  returns a negative integer if this enumerated constant comes before `other`, zero if `this == other`, and a positive integer otherwise. The ordering of the constants is given by the `enum` declaration.

## 5.8. Sealed Classes

Unless a class is declared final, anyone can form a subclass of it. What if you want to have more control? For example, suppose you need to write your own JSON library because none of the existing ones does exactly what you need.

The JSON standard says that a JSON value is an array, number, string, Boolean value, object, or null. An obvious approach is to model this with classes `JSONArray`, `JSONNumber`, and so on that extend an abstract class `JSONValue`:

```
public abstract class JSONValue {
    // Methods that apply to all JSON values
}

public final class JSONArray extends JSONValue {
    . . .
}

public final class JSONNumber extends JSONValue {
    . . .
}
```

By declaring the classes `JSONArray`, `JSONNumber`, and so on as `final`, we can ensure that nobody forms a subclass. But we cannot stop anyone from forming another subclass of `JSONValue`.

Why might we want that control? Consider this code:

```
JSONValue v = . . .;
if (v instanceof JSONArray a) . . .
else if (v instanceof JSONNumber n) . . .
else if (v instanceof JSONString s) . . .
else if (v instanceof JSOBoolean b) . . .
else if (v instanceof JSONObject o) . . .
else . . . // Must be JSONNull
```

Here, the control flow implies that we know all direct subclasses of `JSONValue`. This is not an open-ended hierarchy. The JSON standard won't change; if it does, we as the library implementors will add a seventh subclass. We don't want anyone else out there mess with the class hierarchy.

In Java, a *sealed* class controls which classes may inherit from it. Sealed classes were added as a preview feature in Java 15 and finalized in Java 17.

Here is how to declare the `JSONValue` class as sealed:

```
public abstract sealed class JSONValue
        permits JSONArray, JSONNumber, JSONString, JSONBoolean, JSONObject, JSONNull {
    . . .
}
```

It is an error to define a nonpermitted subclass:

```
public class JSONComment extends JSONValue { . . . } // Error
```

That's just as well, since JSON doesn't allow for comments. Sealed classes thus allow for accurate modeling of domain constraints.

The permitted subclasses of a sealed class must be accessible. They cannot be private classes that are nested in another class, or package-visible classes from another package.

For permitted subclasses that are public, the rules are more stringent. They must be in the same package as the sealed class. However, if you use modules (see **Chapter 12**), then they must only be in the same module.

---



**Note:** A sealed class can be declared without a `permits` clause. Then all of its direct subclasses must be declared in the same file. Programmers without access to that file cannot form subclasses.

A file can have at most one `public` class, so this arrangement appears to be only useful if the subclasses are not for use by the public.

However, as you will see in the next chapter, you can use inner classes as public subclasses.

---

An important motivation for sealed classes is compile-time checking. Consider this method of the `JSONValue` class, which uses a `switch` expression with pattern matching (see **Section 5.9**):

```
public String type() {
    return switch (this) {
        case JSONArray _ -> "array";
        case JSONNumber _ -> "number";
        case JSONString _ -> "string";
        case JSONBoolean _ -> "boolean";
        case JSONObject _ -> "object";
        case JSONNull _ -> "null";
        // No default needed here
    };
}
```

The compiler can check that no `default` clause is needed since all direct subclasses of `JSONValue` occur as cases.

---



**Note:** The preceding `type` method doesn't look very object-oriented. It would be in the spirit of OOP to have each of the six classes provide its own `type` method, relying on polymorphism instead of a switch. For an open-ended hierarchy, that is a good approach. But when there is a fixed set of classes, it is often more convenient to have all alternatives in one method.

---

At first glance, it appears as if a subclass of a sealed class must be final. But for exhaustiveness testing, we only need to know all direct subclasses. It is not a problem if those classes have further subclasses. For example, we can reorganize our JSON hierarchy as shown in **Figure 5.3**.

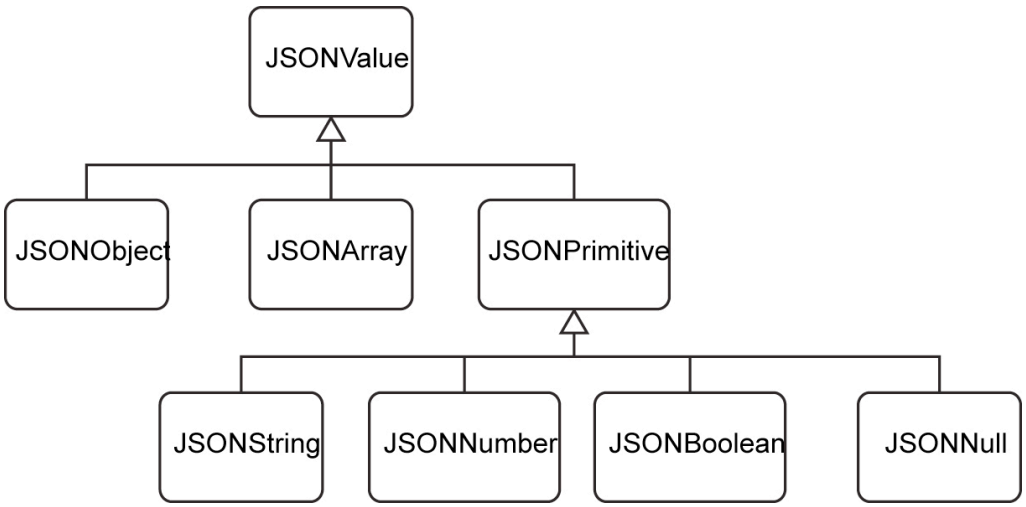



**Figure 5.3:** The complete hierarchy of classes for representing JSON values

In this hierarchy, `JSONValue` permits three subclasses:

```
public abstract sealed class JSONValue permits JSONObject, JSONArray, JSONPrimitive {
    . . .
}
```

The `JSONPrimitive` class is sealed as well:

```
public abstract sealed class JSONPrimitive extends JSONValue
        permits JSONString, JSONNumber, JSONBoolean, JSONNull {
    . . .
}
```

A subclass of a sealed class must specify whether it is `sealed`, `final`, or open for subclassing. In the latter case, it must be declared as `non-sealed`.

---



**Note:** The `non-sealed` keyword is the first Java keyword with a hyphen. This may well be a future trend. Adding keywords to the language always

comes with a risk. Existing code may no longer compile. For that reason, `sealed` is a "contextual" keyword. You can still declare variables or methods named `sealed`:

```
int sealed = 1; // OK to use contextual keyword as identifier
```

With a hyphenated keyword, one doesn't have to worry about this. The only ambiguity is with subtraction:

```
int non = 0;
non = non-sealed; // Subtraction, not keyword
```

---

Why would you ever want a `non-sealed` subclass? Consider an XML node class with six direct subclasses:

```
public abstract sealed class Node permits Element, Text, Comment,
        CDATASection, EntityReference, ProcessingInstruction {
    . . .
}
```

We allow arbitrary subclasses of `Element`:

```
public non-sealed class Element extends Node {
    . . .
}

public class HTMLDivElement extends Element {
    . . .
}
```

In this section, you learned about sealed *classes*. In the next chapter, you will learn about *interfaces*, a generalization of abstract classes. Java interfaces can also have subtypes. Sealed interfaces work exactly the same as sealed classes, controlling the direct subtypes.

The following sample program fleshes out the JSON hierarchy. The implementation of `JSONObject` uses a `HashMap`, which will be covered in **Chapter 9**. In the example, we use interfaces instead of abstract classes so that `JSONNumber` and `JSONString` can be records and the `JSONBoolean` and `JSONNull` classes can be enumerations. Records and enumerations can implement interfaces, but they cannot extend classes.

**Listing 5.12 v1ch05/SealedDemo.java**

```
package v1ch05;

import module java.base;

sealed interface JSONValue permits JSONArray, JSONObject, JSONPrimitive {
    public default String type() {
        if (this instanceof JSONArray) return "array";
        else if (this instanceof JSONObject) return "object";
        else if (this instanceof JSONNumber) return "number";
        else if (this instanceof JSONString) return "string";
        else if (this instanceof JSONBoolean) return "boolean";
        else return "null";
    }
}

final class JSONArray extends ArrayList<JSONValue> implements JSONValue {
}

final class JSONObject extends HashMap<String, JSONValue> implements JSONValue {
    public String toString() {
        StringBuilder result = new StringBuilder();
        result.append("{");
        for (Map.Entry<String, JSONValue> entry : entrySet()) {
            if (result.length() > 1) result.append(",");
            result.append(" \"");
            result.append(entry.getKey());
            result.append("\": ");
            result.append(entry.getValue());
        }
        result.append(" }");
        return result.toString();
    }
}

sealed interface JSONPrimitive extends JSONValue
    permits JSONNumber, JSONString, JSONBoolean, JSONNull {
}

final record JSONNumber(double value) implements JSONPrimitive {
    public String toString() {
        return "" + value;
    }
}

final record JSONString(String value) implements JSONPrimitive {
    public String toString() {
        return "\"" + value.translateEscapes() + "\"";
    }
}

enum JSONBoolean implements JSONPrimitive {
```

```
    FALSE, TRUE;

    public String toString() {
        return super.toString().toLowerCase();
    }
}

enum JSONNull implements JSONPrimitive {
    INSTANCE;

    public String toString() {
        return "null";
    }
}

class SealedDemo {
    void main() {
        JSONObject obj = new JSONObject();
        obj.put("name", new JSONString("Harry"));
        obj.put("salary", new JSONNumber(90000));
        obj.put("married", JSONBoolean.FALSE);
        JSONArray arr = new JSONArray();
        arr.add(new JSONNumber(13));
        arr.add(JSONNull.INSTANCE);

        obj.put("luckyNumbers", arr);
        IO.println(obj);
        IO.println(obj.type());
    }
}
```

## 5.9. Pattern Matching

You have already seen how to use the `instanceof` operator to check whether an object has a particular type. The most convenient form of `instanceof` declares a variable of the matching type:

```
Employee e = . . .;
String description;
if (e instanceof Executive exec)
    description = "An executive with a title of " + exec.getTitle();
else if (e instanceof Manager)
    description = "A manager who deserves a bonus";
else
    description = "A lowly employee with a salary of " + e.getSalary();
```

The pattern-matching form of `switch` simplifies this type analysis:

```
Employee e = . . .;
String description = switch (e) {
    case Executive exec -> "An executive with a title of " + exec.getTitle();
    case Manager _ -> "A manager who deserves a bonus";
    default -> "A lowly employee with a salary of " + e.getSalary();
};
```

Note the symmetry between the clauses `e instanceof Executive exec` and `case Executive exec`. Both declare a variable of the matched type. You can use that variable to access subclass methods (such as `getTitle` in our example).

Starting with Java 22, you can use an underscore if you don't need the variable, as was done in the clause `case Manager _`.



**Note:** Generally, you cannot combine multiple type patterns in the same case, but you can when they don't have pattern variables:

```
case Employee _, Manager _ -> "Not an executive" // Ok to combine these patterns
```

Using the expression form of `switch` is simpler than a chain of `if` statements. The `switch` expression yields a value that is assigned to the `description` variable. In contrast, when using `instanceof`, each branch of the conditional statements must set the `description` value separately.

It is possible to use the statement and fallthrough forms of `switch` with type patterns. But the expression form without fallthrough is usually the most natural choice.

As you have just seen, the basic structure of pattern matching is straightforward. The following subsections go into technical details, some of which are a bit gnarly for compatibility reasons.



**Preview:** Java 25 introduces primitive type patterns as a preview feature. For example,

```
case byte b -> . . .;
```

matches if the selector is an integer between -128 and 127. Then the variable `b` of type `byte` is set to the selector value.

### 5.9.1. Null Handling

The classic `switch` statement is `null`-hostile. If the selector expression is `null`, the `switch` throws a `NullPointerException`. You can avoid this by adding a `case null`:

```
String description = switch (e) {
    case Executive exec -> "An executive with a title of " + exec.getTitle();
    case Manager m -> "A manager who deserves a bonus";
    case null -> "No employee";
    default -> "A lowly employee with a salary of " + e.getSalary();
};
```

You can combine `case null` with `default`, but not with any other case:

```
case null, default -> "No free parking";
```



**Note:** You cannot use `case _` to express a default case. That was done on purpose, so that you don't have to remember whether such a case covers `null` or not.

### 5.9.2. Guards

You can add a condition to a pattern called a *guard*, using the restricted `when` keyword.

```
case Executive exec when exec.getTitle().getLength() >= 20 ->
    "An executive with an impressive title";
```

In the guard expression, you can use the variable that the pattern introduces.

If the guard expression is not fulfilled, the case is not selected and the next case is tested.

If an exception occurs in the `when` clause, the `switch` throws that exception.

Guards are particularly useful in record patterns:

```
Point p = . . .;
String description = switch (p) {
    case Point(var x, var y) when x == 0 && y == 0 -> "origin";
    case Point(var x, _) when x == 0 -> "on x-axis";
    case Point(_, var y) when y == 0 -> "on y-axis";
    default -> "not on either axis";
};
```



**Caution:** A record pattern invokes the component accessors. In the unlikely case that one of them throws an exception, the `switch` throws a `MatchException`. This can only happen for custom accessors.

### 5.9.3. Exhaustiveness

Any `switch` *expression* must be *exhaustive*: there must be a case for each possible selector value. This is an obvious requirement since the expression must always yield a value.

There are several ways for a `switch` to be exhaustive:

- Cases for every instance of an `enum`
- Unguarded type patterns that cover every subtype of a sealed type
- An unguarded type pattern of the selector type
- A `default` clause

Guarded patterns cannot be used for exhaustiveness checks since the compiler cannot evaluate them. For example,

```
case Executive exec when exec.getTitle().getLength() >= 20 -> . . .
case Executive exec when exec.getTitle().getLength() < 20 -> . . .
```

is not exhaustive. You need to rewrite the second clause as

```
case Executive exec -> . . .
```

or

```
default -> . . .
```

---



**Note:** When a sealed type hierarchy or an enumeration class is modified, then a previously exhaustive `switch` might become non-exhaustive. If no cases match, the `switch` throws a `MatchException`.

---

A classic `switch` *statement* need not be exhaustive. If no cases match, then no action takes place. But for greater clarity, all *enhanced* `switch` statements must be exhaustive. A `switch` statement is enhanced if it has a pattern, `case null`, or a selector type other than a primitive type, primitive type wrapper, `String`, or `enum`.

If you don't want to do anything when there is no match, add

```
default -> {} // No fallthrough
```

or

```
default: break; // Fallthrough
```



**Caution:** Testing whether the selector value is `null` is not required for exhaustiveness. You can have an exhaustive `switch` without `case null`. It throws a `NullPointerException` when invoked with a `null` selector.



**Caution:** The absence of `null` checking for exhaustiveness is problematic for nested records. Consider

```
record Box<T>(T contents) { }

Box<Box<String>> doubleBoxed = . . .;
String unboxed = switch (doubleBoxed) {
    case Box(Box(String s)) -> s;
};
```

If `doubleBoxed` is `null`, the `switch` throws a `NullPointerException`. Conversely, if `doubleBoxed` is `new Box<>(new Box<>(null))`, the result of the expression is `null`.

However, if `doubleBoxed` is initialized with `new Box<>(null)`, then a `MatchException` occurs. There should have been a case such as

```
case Box(b) where b == null
```

but the exhaustiveness checker ignores `null`.

### 5.9.4. Dominance

In a classic `switch` with only constant cases, it is illegal to have the same constant in two cases. Therefore, the branches are disjoint and their order does not matter. However, when the cases are patterns, then the order matters. The cases are checked in order, and the first matching case is followed.

When the compiler finds that a case can never match because of a preceding one, a compile-time error occurs. Here is an example:

```
Employee e = . . .;
String description = switch (e) {
    case Employee _ -> "A lowly employee with a salary of " + e.getSalary();
    case Manager m -> "A manager who deserves a bonus"; // ERROR
};
```

Since every `Manager` is an `Employee`, the second case will never match. The first case *dominates* it.

This dominance checking is useful for finding programming errors, but it is not perfect. Here are the fine points that you need to worry about:

- Guarded patterns are not used for dominance checking. There is no way for the compiler to reason about the guards. For example, the compiler cannot tell that

  ```
  case Executive exec when exec.getTitle().getLength() >= 20 -> . . .
  ```

  dominates

  ```
  case Executive exec when exec.getTitle().getLength() >= 30 -> . . .
  ```

- `default` does not dominate constant cases.
- `default` dominates `case null`, even though it does not cover it.

---



**Tip:** While it is possible to place `default` before constant cases, it is a poor idea. Always put it last.

---

### 5.9.5. Patterns and Constants

You cannot match against constants inside record patterns. For example,

```
case Point(0, y) // ERROR
```

is wrong. Use a guard:

```
case Point(x, y) when x == 0 // Ok
```

Numeric constant patterns are only allowed when the *selector* type is `int`, `short`, `char`, or `byte`, or their wrapper types. String constant patterns are only allowed for a selector of type `String`. For example, the following is not legal:

```
Object obj = . . .;
String description = switch (obj) {
    case 0 -> "zero"; // ERROR
    case "" -> "empty string"; // ERROR
    case null -> "null";
    default -> obj.toString();
};
```

Since the selector type is `Object`, the `switch` cannot have numeric or string constant patterns. The remedy is:

```
case Integer i when i == 0
case String s when s.equals("null")
```

The rules are different for `enum` constants. The selector type must merely be a supertype of the enumerated type; that is, `Object`, `Enum`, or an interface (see **Chapter 6**):

```
Object obj = . . .;
String description = switch (obj) {
    case DayOfWeek.FRIDAY -> "TGIF"; // Ok
    . . .
};
```

### 5.9.6. Variable Scope and Fallthrough

This section covers subtle points about variables when execution falls through from one `case` to the next. If you never use fallthrough, just move on to the next section.

When a type or record pattern introduces a variable, its scope extends to the end of the `case`, even if execution falls through the next case. For example, consider the variable `s` in the following code snippet:

```
Object obj = . . .;
switch (obj) {
    case String s when s.length() == 0:
        IO.println("empty string"); // Scope ends here
    default:
        IO.println(obj.toString());
}
```

You can use `s` in the guard clause and the statements following the `:`, but not in the next case. This makes sense since execution can start in that case when `obj` is not a `String`.

You cannot fall into a type pattern:

```
switch (obj) {
    case String s when s.length() > 0:
        IO.println(s.codePointAt(0)); // Fallthrough
    case Integer n:
        IO.println(n); // ERROR
    . . .
}
```

If execution fell from the first case to the body of the second case, `n` could not be bound.

You *can* fall into a type pattern with an unnamed variable:

```
switch (obj) {
    case String s when s.length() == 0:
        IO.print("empty "); // Fallthrough
    case String _:
        IO.println("string"); break; // Ok
    . . .
}
```



**Caution:** Ever since Java 1.0, it has been legal to declare a variable in the body of a `case`. The scope of that variable extends to the end of the `switch`. This scope has never been useful since it leads to an error if you access the variable outside the `case`:

```
switch (obj) {
    case String s:
        String t = s; // OK
    default:
        IO.println(t); // ERROR—t may not have been initialized
}
```

The programs in **Listing 5.13** and **Listing 5.14** demonstrate pattern matching with type and record patterns.

**Listing 5.13 v1ch05/TypePatternDemo.java**

```
1  package v1ch05;
2
3  import com.horstmann.corejava.*;
4
5  class TypePatternDemo {
6      void main() {
7          int r = (int) (4 * Math.random());
8          Employee e = switch (r) {
9              case 0 -> new Employee("Harry Hacker", 50000, 1989, 10, 1);
10             case 1 -> new Manager("Carl Cracker", 80000, 1987, 12, 15);
11             case 2 -> new Executive("Sue Striver", "Senior Associate Vice Preside
12                     200000, 1995,1, 20);
13             default -> null;
14         };
15         String description = switch (e) {
16             case Executive exec when exec.getTitle().length() >= 20
17                     -> "An executive with an impressive title";
18             case Executive exec
19                     -> "An executive with a title of " + exec.getTitle();
```

```
            case Manager m -> {
                    m.setBonus(10000);
                    yield "A manager who just got a bonus";
                }
            case null -> "No employee";
            default -> "A lowly employee with a salary of " + e.getSalary();
        };
        IO.println(description);
    }
}
```

**Listing 5.14 v1ch05/RecordPatternDemo.java**

```
package v1ch05;

record Point(double x, double y) {
}

class RecordPatternDemo {
    void main() {
        int r = (int) (4 * Math.random());
        Point p = switch (r) {
            case 0 -> new Point(0, 0);
            case 1 -> new Point(1, 0);
            case 2 -> new Point(0, 1);
            default -> new Point(1, 1);
        };
        String description = switch (p) {
            case Point(var x, var y) when x == 0 && y == 0 -> "origin";
            case Point(var x, var _) when x == 0 -> "on x-axis";
            case Point(var _, var y) when y == 0 -> "on y-axis";
            default -> "not on either axis";
        };
        IO.println("%s %s%n".formatted(p, description));
    }
}
```

## 5.10. Reflection

The *reflection library* gives you a very rich and elaborate toolset to write programs that manipulate Java code dynamically. Using reflection, Java can support user interface builders, object-relational mappers, and many other development tools that dynamically inquire about the capabilities of classes.

A program that can analyze the capabilities of classes is called *reflective*. The reflection mechanism is extremely powerful. As the next sections show, you can use it to

- Analyze the capabilities of classes at runtime

- Inspect objects at runtime—for example, to write a single `toString` method that works for *all* classes
- Implement generic array manipulation code
- Take advantage of `Method` objects that work just like function pointers in languages such as C++

Reflection is a powerful and complex mechanism; however, it is of interest mainly to tool builders, not application programmers. If you are interested in programming applications rather than tools for other Java programmers, you can safely skip the remainder of this chapter and return to it later.

### 5.10.1. The `Class` Class

While your program is running, the Java runtime system always maintains what is called *runtime type identification* on all objects. This information keeps track of the class to which each object belongs. Runtime type information is used by the virtual machine to select the correct methods to execute.

However, you can also access this information by working with a special Java class. The class that holds this information is called, somewhat confusingly, `Class`. The `getClass()` method in the `Object` class returns an instance of `Class` type.

```
Employee e;
. . .
Class cl = e.getClass();
```

Just like an `Employee` object describes the properties of a particular employee, a `Class` object describes the properties of a particular class. Probably the most commonly used method of `Class` is `getName`. This returns the name of the class. For example, the statement

```
IO.println(e.getClass().getName() + " " + e.getName());
```

prints

```
Employee Harry Hacker
```

if `e` is an employee, or

```
Manager Harry Hacker
```

if `e` is a manager.

If the class is in a package, the package name is part of the class name:

```
var now = new Date();
Class cl = now.getClass();
String className = cl.getName(); // name is set to "java.util.Date"
```

You can obtain a `Class` object corresponding to a class name by using the static `forName` method.

```
String className = "java.util.Date";
Class cl = Class.forName(className);
```

Use this method if the class name is stored in a string that varies at runtime. This works if `className` is the name of a class or interface. Otherwise, the `forName` method throws a *checked exception*. See **Section 5.10.2** for how to supply an *exception handler* whenever you use this method.

A third method for obtaining an object of type `Class` is a convenient shorthand. If `T` is any Java type (or the `void` keyword), then `T.class` is the matching class object. For example:

```
Class cl1 = Date.class; // if you import java.util.*;
Class cl2 = int.class;
Class cl3 = Double[].class;
```

Note that a `Class` object really describes a *type*, which may or may not be a class. For example, `int` is not a class, but `int.class` is nevertheless an object of type `Class`.



**Note:** The `Class` class is actually a generic class. For example, `Employee.class` is of type `Class<Employee>`. In this chapter, I'm not dwelling on this issue because it would further complicate an already abstract concept. For most practical purposes, you can ignore the type parameter and work with the raw `Class` type. See **Chapter 8** for more information on the type parameter of `Class`.



**Caution:** For historical reasons, the `getName` method returns somewhat strange names for array types:

- `Double[].class.getName()` returns `"[Ljava.lang.Double;"`.
- `int[].class.getName()` returns `"[I"`.

The virtual machine manages a unique `Class` object for each type. Therefore, you can use the `==` operator to compare class objects. For example:

```
if (e.getClass() == Employee.class) . . .
```

This test passes if `e` is an instance of `Employee`. Unlike the condition `e instanceof Employee`, this test fails if `e` is an instance of a subclass such as `Manager`.



**Caution:** The `getClass` test fails for anonymous subclasses, which you will see in **Chapter 6**. Actually, you have already encountered one such case. If you override a method in an `enum` instance, then it belongs to a different class:

```
public enum Size {
    SMALL,
    MEDIUM,
    LARGE,
    EXTRA_LARGE {
       public String toString() { return "XL"; }
    };
}
```

Here `Size.EXTRA_LARGE.getClass() != Size.class`.

If you have an object of type `Class`, you can use it to construct instances of the class. Call the `getConstructor` method to get an object of type `Constructor`, then use the `newInstance` method to construct an instance. For example:

```
var className = "java.util.Date";
    // or any other name of a class with a no-arg constructor
Class cl = Class.forName(className);
Object obj = cl.getConstructor().newInstance();
```

If the class doesn't have a constructor without arguments, the `getConstructor` method throws an exception. You will see in **Section 5.10.7** how to invoke other constructors.



**Note:** There is a `Class.newInstance` method that also constructs an instance with the no-argument constructor. However, it is deprecated for a subtle reason. Any exception thrown in the constructor is rethrown as is. For checked exceptions, this violates the compile-time checking. In contrast, `Constructor.newInstance` wraps any constructor exception into an `InvocationTargetException`.



**Note:** The `Class` class is similar to the `type_info` class in C++ or the `type` class in Python. The Java `Class` is quite a bit more versatile than its

analog in C++, but less flexible than in Python, where you can modify classes after they have been declared.

`java.lang.Class` 1.0

- `static Class forName(String className)`
  returns the `Class` object representing the class with name `className`.
- `Constructor getConstructor(Class... parameterTypes)` 1.1
  yields an object describing the constructor with the given parameter types. See for more information on how to supply parameter types.

`java.lang.reflect.Constructor` 1.1

- `Object newInstance(Object... initargs)`
  constructs a new instance of the constructor's declaring class, passing `initargs` to the constructor. See for more information on how to supply arguments.

`java.lang.Throwable` 1.0

- `void printStackTrace()`
  prints the `Throwable` object and the stack trace to the standard error stream.

### 5.10.2. A Primer on Declaring Exceptions

I cover exception handling fully in **Chapter 7**, but in the meantime you will occasionally need to declare methods that can throw exceptions.

When an error occurs at runtime, a program can "throw an exception." Throwing an exception is more flexible than terminating the program because you can provide a *handler* that "catches" the exception and deals with it.

If you don't provide a handler, the program terminates and prints a message to the console, giving the type of the exception. You may have already seen exception reports when you accidentally used a `null` reference or overstepped the bounds of an array.

There are two kinds of exceptions: *unchecked* exceptions and *checked* exceptions. With checked exceptions, the compiler checks that you, the programmer, are aware of the exception and are prepared to deal with the consequences. However, many common exceptions, such as bounds errors, or accessing a `null` reference, are unchecked. The compiler does not expect that you provide a handler—after all, you should spend your mental energy on avoiding these mistakes rather than coding handlers for them.

But not all errors are avoidable. If an exception can occur despite your best efforts, then most Java APIs will throw a checked exception. One example is the `Class.forName` method. There is no way for you to ensure that a class with the given name exists. In **Chapter 7**, you will see several strategies for exception handling. For now, I just show you the simplest strategy.

Whenever a method contains a statement that might throw a checked exception, add a `throws Exception` clause to the method name.

```
public static void doSomethingWithClass(String name)
      throws Exception {
    Class cl = Class.forName(name); // might throw exception
    do something with cl
}
```

Any method that calls this method also needs a `throws Exception` declaration. This includes the `main` method. If an exception actually occurs, the `main` method terminates with a stack trace. (You will learn in **Chapter 7** how to catch exceptions instead of having them terminate your programs.)

You only need to supply a `throws Exception` clause for checked exceptions. It is easy to find out which methods throw checked exceptions—the compiler will complain whenever you call a method that threatens to throw a checked exception.

### 5.10.3. Resources

Classes often have associated data files, such as:

- Image and sound files
- Text files with message strings and button labels

In Java, such an associated file is called a *resource*.

For example, consider a dialog box that displays a message such as the one in **Figure 5.4**.




**Figure 5.4:** Displaying image and text resources

Of course, the book title and copyright year in the panel will change for the next edition of the book. To make it easy to track this change, we will put the text inside a file and not hardcode it as a string.

But where should you put a file such as `about.txt`? Of course, it would be convenient to simply place it with the rest of the program files inside a JAR file.

The `Class` class provides a useful service for locating resource files. Here are the necessary steps:

1. Get the `Class` object of the class that has a resource—for example, `ResourceDemo.class`.
2. Some methods, such as the `getImage` method of the `ImageIcon` class, accept URLs that describe resource locations. Then you call

   ```
   URL url = cl.getResource("about.png");
   ```

3. Otherwise, use the `getResourceAsStream` method to obtain an input stream for reading the data in the file.

The point is that the Java virtual machine knows how to locate a class, so it can then search for the associated resource *in the same location*. For example, suppose the `ResourceDemo` class is in a package `resources`. Then the `ResourceDemo.class` file is located in a `resources` directory, and you place an icon file into the same directory.

Instead of placing a resource file inside the same directory as the class file, you can provide a relative or absolute path such as

```
data/about.txt
/images/about.gif
```

Automating the loading of files is all the resource loading feature does. There are no standard methods for interpreting the contents of resource files. Each program must have its own way of interpreting its resource files.

Another common application of resources is the internationalization of programs. Language-dependent strings, such as messages and user interface labels, are stored in resource files, with one file per language. The *internationalization API*, which is discussed in **Chapter 7 of Volume II**, supports a standard method for organizing and accessing these localization files.

**Listing 5.15** is a program that demonstrates resource loading. (Do not worry about the code for reading text and displaying dialogs—we cover those details later.) Compile, build a JAR file, and execute it:

```
javac v1ch05/resources/ResourceDemo.java
jar cvfe ResourceDemo.jar v1ch05.resources.ResourceDemo \
    v1ch05/resources/*.class images/*.gif v1ch05/resources/data/*.txt
java -jar ResourceDemo.jar
```

Move the JAR file to a different directory and run it again to check that the program reads the resource files from the JAR file, not from the current directory.

**Listing 5.15 v1ch05/resources/ResourceDemo.java**

```
1  package v1ch05.resources;
2
3  import module java.base;
4  import module java.desktop;
5
6  /**
7   */
8  class ResourceDemo {
9      void main() throws Exception {
10         URL aboutURL = getClass().getResource("/images/about.gif");
11         var icon = new ImageIcon(aboutURL);
12
13         InputStream stream = getClass().getResourceAsStream("data/about.txt");
14         var about = new String(stream.readAllBytes());
15
16         InputStream stream2 = getClass().getResourceAsStream("data/title.txt");
17         var title = new String(stream2.readAllBytes()).strip();
18
19         JOptionPane.showMessageDialog(null, about, title, JOptionPane.INFORMATION
20                 icon);
21     }
22 }
```

`java.lang.Class` 1.0

- `URL getResource(String name)` 1.1
- `InputStream getResourceAsStream(String name)` 1.1

  find the resource in the same place as the class and then return a URL or input stream that you can use for loading the resource. Return `null` if the resource isn't found, so do not throw an exception for an I/O error.

### 5.10.4. Using Reflection to Analyze the Capabilities of Classes

Here is a brief overview of the most important parts of the reflection mechanism for letting you examine the structure of a class.

The three classes `Field`, `Method`, and `Constructor` in the `java.lang.reflect` package describe the fields, methods, and constructors of a class, respectively. All three classes have a method called `getName` that returns the name of the item. The `Field` class has a method `getType` that returns an object, again of type `Class`, that describes the field type. The `Method` and `Constructor` classes have methods to report the types of the parameters, and the `Method` class also reports the return type.

All three of these classes also have a method called `getModifiers` that returns an integer, with various bits turned on and off, that describes the modifiers used, such as `public` and `static`. You can then use the static methods in the `Modifier` class in the `java.lang.reflect` package to analyze the integer that `getModifiers` returns. Use methods like `isPublic`, `isPrivate`, or `isFinal` in the `Modifier` class to tell whether a method or constructor was `public`, `private`, or `final`. All you have to do is have the appropriate method in the `Modifier` class work on the integer that `getModifiers` returns. You can also use the `Modifier.toString` method to print the modifiers.

Alternatively, the `accessFlags` method returns a set of values of the `AccessFlag` enumeration. For example, here is how you can test whether a method is static:

```
if (Modifiers.isStatic(m.getModifiers())) . . .
if (m.accessFlags().contains(AccessFlag.STATIC)) . . .
```

The `getFields`, `getMethods`, and `getConstructors` methods of the `Class` class return arrays of the *public* fields, methods, and constructors that the class supports. This includes public members of superclasses. The `getDeclaredFields`, `getDeclaredMethods`, and `getDeclaredConstructors` methods of the `Class` class return arrays consisting of all fields, methods, and constructors that are declared in the class. This includes private, package, and protected members, as well as members with package access, but not members of superclasses.

Listing 5.16 shows you how to print out all information about a class. The program prompts you for the name of a class and writes out the signatures of all methods and constructors as well as the names of all instance fields of a class. For example, if you enter

```
java.lang.Double
```

the program prints

```
public final class java.lang.Double extends java.lang.Number {
    public java.lang.Double(double);
    public java.lang.Double(java.lang.String);

    public boolean equals(java.lang.Object);
    public static java.lang.String toString(double);
    public java.lang.String toString();
    public static int hashCode(double);
    public int hashCode();
    public static double min(double, double);
    public static double max(double, double);
    public static native long doubleToRawLongBits(double);
    // more methods

    public static final double POSITIVE_INFINITY;
    public static final double NEGATIVE_INFINITY;
    public static final double NaN;
    public static final double MAX_VALUE;
    public static final double MIN_NORMAL;
    public static final double MIN_VALUE;
    private final double value;
    // more fields
}
```

What is remarkable about this program is that it can analyze any class that the Java interpreter can load, not just the classes that were available when the program was compiled. We will use this program in the next chapter to peek inside the inner classes that the Java compiler generates automatically.

**Listing 5.16 v1ch05/ReflectionDemo.java**

```
1  package v1ch05;
2
3  import module java.base;
4
5  /**
6   * This program uses reflection to print all features of a class.
7   */
8  class ReflectionDemo {
9      void main(String[] args) throws Exception {
```

```
        // read class name from command line args or user input
        String name = args.length > 0 ? args[0] :
            IO.readln("Enter class name (e.g. java.util.Date): ");

        // print class modifiers, name, and superclass name (if != Object)
        Class cl = Class.forName(name);
        String modifiers = Modifier.toString(cl.getModifiers());
        if (modifiers.length() > 0) IO.print(modifiers + " ");
        if (cl.isSealed()) IO.print("sealed ");
        if (cl.isEnum())
            IO.print("enum " + name);
        else if (cl.isRecord())
            IO.print("record " + name);
        else if (cl.isInterface())
            IO.print("interface " + name);
        else
            IO.print("class " + name);
        Class supercl = cl.getSuperclass();
        if (supercl != null && supercl != Object.class)
            IO.print(" extends " + supercl.getName());

        printInterfaces(cl);
        printPermittedSubclasses(cl);

        IO.print(" {\n");
        printConstructors(cl);
        IO.println();
        printMethods(cl);
        IO.println();
        printFields(cl);
        IO.println("}");
    }

    /**
     * Prints all constructors of a class
     * @param cl a class
     */
    public static void printConstructors(Class cl) {
        Constructor[] constructors = cl.getDeclaredConstructors();

        for (Constructor c : constructors) {
            String name = c.getName();
            IO.print("    ");
            String modifiers = Modifier.toString(c.getModifiers());
            if (modifiers.length() > 0) IO.print(modifiers + " ");
            IO.print(name + "(");

            // print parameter types
            Class[] paramTypes = c.getParameterTypes();
            for (int j = 0; j < paramTypes.length; j++) {
                if (j > 0) IO.print(", ");
                IO.print(paramTypes[j].getName());
            }
            IO.println(");");
```

```
64          }
65      }
66
67      /**
68       * Prints all methods of a class
69       * @param cl a class
70       */
71      public static void printMethods(Class cl) {
72          Method[] methods = cl.getDeclaredMethods();
73
74          for (Method m : methods) {
75              Class retType = m.getReturnType();
76              String name = m.getName();
77
78              IO.print("    ");
79              // print modifiers, return type and method name
80              String modifiers = Modifier.toString(m.getModifiers());
81              if (modifiers.length() > 0) IO.print(modifiers + " ");
82              IO.print(retType.getName() + " " + name + "(");
83
84              // print parameter types
85              Class[] paramTypes = m.getParameterTypes();
86              for (int j = 0; j < paramTypes.length; j++) {
87                  if (j > 0) IO.print(", ");
88                  IO.print(paramTypes[j].getName());
89              }
90              IO.println(");");
91          }
92      }
93
94      /**
95       * Prints all fields of a class
96       * @param cl a class
97       */
98      public static void printFields(Class cl) {
99          Field[] fields = cl.getDeclaredFields();
100
101         for (Field f : fields) {
102             Class type = f.getType();
103             String name = f.getName();
104             IO.print("    ");
105             String modifiers = Modifier.toString(f.getModifiers());
106             if (modifiers.length() > 0) IO.print(modifiers + " ");
107             IO.println(type.getName() + " " + name + ";");
108         }
109     }
110
111     /**
112      * Prints all permitted subtypes of a sealed class
113      * @param cl a class
114      */
115     public static void printPermittedSubclasses(Class cl) {
116         if (cl.isSealed()) {
117             Class<?>[] permittedSubclasses = cl.getPermittedSubclasses();
```

```
118             for (int i = 0; i < permittedSubclasses.length; i++) {
119                 if (i == 0)
120                     IO.print(" permits ");
121                 else
122                     IO.print(", ");
123                 IO.print(permittedSubclasses[i].getName());
124             }
125         }
126     }
127
128     /**
129      * Prints all directly implemented interfaces of a class
130      * @param cl a class
131      */
132     public static void printInterfaces(Class cl) {
133         Class<?>[] interfaces = cl.getInterfaces();
134         for (int i = 0; i < interfaces.length; i++) {
135             if (i == 0)
136                 IO.print(cl.isInterface() ? " extends " : " implements ");
137             else
138                 IO.print(", ");
139             IO.print(interfaces[i].getName());
140         }
141     }
142 }
```

`java.lang.Class` 1.0

- `Field[] getFields()` 1.1
- `Field[] getDeclaredFields()` 1.1

`getFields` returns an array containing `Field` objects for the public fields of this class or its superclasses; `getDeclaredFields` returns an array of `Field` objects for all fields of this class. The methods return an array of length 0 if there are no such fields or if the `Class` object represents a primitive or array type.

- `Method[] getMethods()` 1.1
- `Method[] getDeclaredMethods()` 1.1

return an array containing `Method` objects: `getMethods` returns public methods and includes inherited methods; `getDeclaredMethods` returns all methods of this class or interface but does not include inherited methods.

- `Constructor[] getConstructors()` 1.1
- `Constructor[] getDeclaredConstructors()` 1.1

return an array containing `Constructor` objects that give you all the public constructors (for `getConstructors`) or all constructors (for `getDeclaredConstructors`) of the class represented by this `Class` object.

- `isInterface()`
  returns `true` if this `Class` object describes an `interface`. (See **Chapter 6** for interfaces.)
- `isEnum()` 1.5
  returns `true` if this `Class` object describes an `enum`.
- `isRecord()` 16
  returns `true` if this `Class` object describes a `record`.
- `RecordComponent[] getRecordComponents()` 16
  returns an array of `RecordComponent` objects that describe the record fields, or `null` if this class is not a record.
- `String getPackageName()` 9
  gets the name of the package containing this type, or the package of the element type if this type is an array type, or `"java.lang"` if this type is a primitive type.

`java.lang.reflect.Field` 1.1

`java.lang.reflect.Method` 1.1

`java.lang.reflect.Constructor` 1.1

- `Class getDeclaringClass()`
  returns the `Class` object for the class that defines this constructor, method, or field.
- `Class[] getExceptionTypes()` (in `Constructor` and `Method` classes)
  returns an array of `Class` objects that represent the types of the exceptions thrown by the method.
- `int getModifiers()`
  returns an integer that describes the modifiers of this constructor, method, or field. Use the methods in the `Modifier` class to analyze the return value.
- `Set<AccessFlag> accessFlags()` 20
  returns a set of values in the `AccessFlag` enumeration that describe the access modifiers of this constructor, method, or field.
- `String getName()`
  returns a string that is the name of the constructor, method, or field.
- `Class[] getParameterTypes()` (in `Constructor` and `Method` classes)
  returns an array of `Class` objects that represent the types of the parameters.
- `Class getReturnType()` (in `Method` class)
  returns a `Class` object that represents the return type.

`java.lang.reflect.RecordComponent` 16

- `String getName()`
- `Class getType()`
  get the name and type of this record component.
- `Method getAccessor()`
  returns the `Method` object for accessing this record component.

`java.lang.reflect.Modifier` 1.1

- `static String toString(int modifiers)`
  returns a string with the modifiers that correspond to the bits set in `modifiers`.
- `static boolean isAbstract(int modifiers)`
- `static boolean isFinal(int modifiers)`
- `static boolean isInterface(int modifiers)`
- `static boolean isNative(int modifiers)`
- `static boolean isPrivate(int modifiers)`
- `static boolean isProtected(int modifiers)`
- `static boolean isPublic(int modifiers)`
- `static boolean isStatic(int modifiers)`
- `static boolean isStrict(int modifiers)`
- `static boolean isSynchronized(int modifiers)`
- `static boolean isVolatile(int modifiers)`
  test the bit in the `modifiers` value that corresponds to the modifier in the method name.

### 5.10.5. Using Reflection to Analyze Objects at Runtime

In the preceding section, we saw how we can find out the *names* and *types* of the instance fields of any object:

- Get the corresponding `Class` object.
- Call `getDeclaredFields` on the `Class` object.

In this section, we will go one step further and actually look at the *contents* of the fields. Of course, it is easy to look at the contents of a specific field of an object whose name and type are known when you write a program. But reflection lets you look at fields of objects that were not known at compile time.

The key method to achieve this is the `get` method in the `Field` class. If `f` is an object of type `Field` (for example, one obtained from `getDeclaredFields`) and `obj` is an object of the class of which `f` is a field, then `f.get(obj)` returns an object whose value is the current value of the field of `obj`. This is all a bit abstract, so let's run through an example.

```
var harry = new Employee("Harry Hacker", 50000, 10, 1, 1989);
Class cl = harry.getClass();
    // the class object representing Employee
Field f = cl.getDeclaredField("name");
    // the name field of the Employee class
Object v = f.get(harry);
    // the value of the name field of the harry object, i.e.,
    // the String object "Harry Hacker"
```

Actually, there is a problem with this code. Since the `name` field is a private field, the `get` and `set` methods will throw an `IllegalAccessException`. You can only use `get` and `set` with accessible fields. The security mechanism of Java lets you find out what fields an object has, but it won't let you read and write the values of those fields unless you have permission.

The default behavior of the reflection mechanism is to respect Java access control. However, you can override access control by invoking the `setAccessible` method on a `Field`, `Method`, or `Constructor` object. For example:

```
f.setAccessible(true); // now OK to call f.get(harry)
```

The `setAccessible` method is a method of the `AccessibleObject` class, the common superclass of the `Field`, `Method`, and `Constructor` classes. This feature is provided for debuggers, persistent storage, and similar mechanisms. We will use it for a generic `toString` method later in this section.

When you run the program with Java 17 or higher, the call to `setAccessible` throws an `InaccessibleObjectException` exception.

To keep the program running, "open" the `java.util` and `java.lang` packages in the `java.base` module to the "unnamed module." The details are in **Chapter 12**. Here is the syntax:

```
java --add-opens java.base/java.util=ALL-UNNAMED \
    --add-opens java.base/java.lang=ALL-UNNAMED \
    v1ch05/objectAnalyzer/ObjectAnalyzerDemo.java
```

Once you have reflective access to an object, you can set any field values. The call `f.set(obj, value)` sets the field represented by `f` of the object `obj` to the new value. This works even for `final` fields! However, record components cannot be set by reflection.



**Note:** It is possible that future libraries will use *variable handles* instead of reflection for reading and writing fields. A `VarHandle` is similar to a `Field`. You can use it to read or write a specific field of any instance of a

specific class. However, to obtain a `VarHandle`, the library code needs a `Lookup` object:

```
public Object getFieldValue(Object obj, String fieldName, Lookup lookup)
        throws NoSuchFieldException, IllegalAccessException {
    Class cl = obj.getClass();
    Field field = cl.getDeclaredField(fieldName);
    VarHandle handle = MethodHandles.privateLookupIn(cl, lookup)
        .unreflectVarHandle(field);
    return handle.get(obj);
}
```

This works provided that the `Lookup` object is generated in the module that has the permission to access the field. There must be a method in the module that calls `MethodHandles.lookup()`, yielding an object encapsulating the access rights of the caller. In this way, one module can give permission for accessing private members to another module. The practical issue is how those permissions can be given with a minimum of hassle.

---

While we can still do so, let us look at a generic `toString` method that works for *any* class (see **Listing 5.17**). The generic `toString` method uses `getDeclaredFields` to obtain all instance fields and the `setAccessible` convenience method to make all fields accessible. For each field, it obtains the name and the value. Each value is turned into a string by recursively invoking `toString`.

The generic `toString` method needs to address a couple of complexities. Cycles of references could cause an infinite recursion. Therefore, the `ObjectAnalyzer` keeps track of objects that were already visited. Also, to peek inside arrays, you need a different approach. You'll learn about the details in the next section.

You can use this `toString` method to peek inside any object. For example, the call

```
var squares = new ArrayList<Integer>();
for (int i = 1; i <= 5; i++) squares.add(i * i);
IO.println(new ObjectAnalyzer().toString(squares));
```

yields the printout

```
java.util.ArrayList[elementData=class java.lang.Object[]{
java.lang.Integer[value=1][][],java.lang.Integer[value=4][][],
java.lang.Integer[value=9][][],java.lang.Integer[value=16][][],
java.lang.Integer[value=25][][], null,null,null,null,null},size=5]
[modCount=5][][]
```

You can use this generic `toString` method to implement the `toString` methods of your own classes, like this:

```
public String toString() {
    return new ObjectAnalyzer().toString(this);
}
```

However, before you get too excited about never having to implement `toString` again, remember that showing arbitrary field values requires flags that can compromise security.

**Listing 5.17 v1ch05/objectAnalyzer/ObjectAnalyzerDemo.java**

```
1  package v1ch05.objectAnalyzer;
2
3  import module java.base;
4
5  /**
6   * This program uses reflection to spy on objects.
7   */
8  class ObjectAnalyzerDemo {
9      void main() throws Exception {
10         var squares = new ArrayList<Integer>();
11         for (int i = 1; i <= 5; i++)
12             squares.add(i * i);
13         IO.println(new ObjectAnalyzer().toString(squares));
14     }
15 }
```

**Listing 5.18 v1ch05/objectAnalyzer/ObjectAnalyzer.java**

```
1  package v1ch05.objectAnalyzer;
2
3  import module java.base;
4
5  public class ObjectAnalyzer {
6      private ArrayList<Object> visited = new ArrayList<>();
7
8      /**
9       * Converts an object to a string representation that lists all fields.
10      * @param obj an object
11      * @return a string with the object's class name and all field names and valu
12      */
13     public String toString(Object obj) throws Exception {
14         if (obj == null) return "null";
15         if (visited.contains(obj)) return "...";
16         visited.add(obj);
17         Class cl = obj.getClass();
18         if (cl == String.class) return (String) obj;
19         if (cl.isArray()) {
```

```
            String r = cl.getComponentType() + "[]{";
            for (int i = 0; i < Array.getLength(obj); i++) {
                if (i > 0) r += ",";
                Object val = Array.get(obj, i);
                if (cl.getComponentType().isPrimitive())
                    r += val;
                else
                    r += toString(val);
            }
            return r + "}";
        }

        String r = cl.getName();
        // inspect the fields of this class and all superclasses
        do {
            r += "[";
            Field[] fields = cl.getDeclaredFields();
            AccessibleObject.setAccessible(fields, true);
            // get the names and values of all fields
            for (Field f : fields) {
                if (!Modifier.isStatic(f.getModifiers())) {
                    if (!r.endsWith("[")) r += ",";
                    r += f.getName() + "=";
                    Class t = f.getType();
                    Object val = f.get(obj);
                    if (t.isPrimitive())
                        r += val;
                    else
                        r += toString(val);
                }
            }
            r += "]";
            cl = cl.getSuperclass();
        }
        while (cl != null);

        return r;
    }
}
```

`java.lang.reflect.AccessibleObject` 1.2

- `void setAccessible(boolean flag)`
  sets or clears the accessibility flag for this accessible object, or throws an `IllegalAccessException` if the access is denied.
- `boolean trySetAccessible()` 9
  sets the accessibility flag for this accessible object, or returns `false` if the access is denied.
- `boolean canAccess(Object obj)` 9
  checks if the caller can access `obj` through this field, method, or constructor object. Pass `null` for a static field or method, or for a

constructor.

- `static void setAccessible(AccessibleObject[] array, boolean flag)`
  is a convenience method to set the accessibility flag for an array of objects.

`java.lang.Class` 1.1

- `Field getField(String name)`
- `Field[] getFields()`
  get the public field with the given name, or an array of all fields.
- `Field getDeclaredField(String name)`
- `Field[] getDeclaredFields()`
  get the field that is declared in this class with the given name, or an array of all fields.

`java.lang.reflect.Field` 1.1

- `Object get(Object obj)`
  gets the value of the field described by this `Field` object in the object `obj`.
- `void set(Object obj, Object newValue)`
  sets the field described by this `Field` object in the object `obj` to a new value.

### 5.10.6. Using Reflection to Write Generic Array Code

The `Array` class in the `java.lang.reflect` package allows you to create arrays dynamically. This is used, for example, in the implementation of the `copyOf` method in the `Arrays` class. Recall how this method can be used to grow an array that has become full.

```
var a = new Employee[100];
. . .
// array is full
a = Arrays.copyOf(a, 2 * a.length);
```

How can one write such a generic method? It helps that an `Employee[]` array can be converted to an `Object[]` array. That sounds promising. Here is a first attempt:

```
public static Object[] badCopyOf(Object[] a, int newLength) // not useful {
    var newArray = new Object[newLength];
    System.arraycopy(a, 0, newArray, 0, Math.min(a.length, newLength));
    return newArray;
}
```

However, there is a problem with actually *using* the resulting array. The type of array that this code returns is an array of *objects* (`Object[]`) because we created the array using the line of code

```
new Object[newLength]
```

An array of objects *cannot* be cast to an array of employees (`Employee[]`). The virtual machine would generate a `ClassCastException` at runtime. The point is that, as mentioned earlier, a Java array remembers the type of its entries—that is, the element type used in the `new` expression that created it. It is legal to cast an `Employee[]` temporarily to an `Object[]` array and then cast it back, but an array that started its life as an `Object[]` array can never be cast into an `Employee[]` array. To write this kind of generic array code, we need to be able to make a new array of the *same* type as the original array. For this, we need the methods of the `Array` class in the `java.lang.reflect` package. The key is the static `newInstance` method of the `Array` class that constructs a new array. You must supply the type for the entries and the desired length as arguments to this method.

```
Object newArray = Array.newInstance(componentType, newLength);
```

To actually carry this out, we need to get the length and the component type of the new array.

We obtain the length by calling `Array.getLength(a)`. The static `getLength` method of the `Array` class returns the length of an array. To get the component type of the new array:

1. First, get the class object of `a`.
2. Confirm that it is indeed an array.
3. Use the `getComponentType` method of the `Class` class (which is defined only for class objects that represent arrays) to find the right type for the array.
4. Conversely, for any `Class` object representing a class C, the `arrayType` method yields the `Class` object representing C`[]`.

Why is `getLength` a method of `Array` but `getComponentType` a method of `Class`? I don't know—the distribution of the reflection methods seems a bit ad hoc at times.

Here's the code:

```
public static Object goodCopyOf(Object a, int newLength) {
    Class cl = a.getClass();
    if (!cl.isArray()) return null;
    Class componentType = cl.getComponentType();
    int length = Array.getLength(a);
```

```
    Object newArray = Array.newInstance(componentType, newLength);
    System.arraycopy(a, 0, newArray, 0, Math.min(length, newLength));
    return newArray;
}
```

Note that this `copyOf` method can be used to grow arrays of any type, not just arrays of objects.

```
int[] a = { 1, 2, 3, 4, 5 };
a = (int[]) goodCopyOf(a, 10);
```

To make this possible, the parameter of `goodCopyOf` is declared to be of type `Object`, *not an array of objects* (`Object[]`). The integer array type `int[]` can be converted to an `Object`, but not to an array of objects!

**Listing 5.19** shows both methods in action. Note that the cast of the return value of `badCopyOf` will throw an exception.

**Listing 5.19 v1ch05/CopyOfDemo.java**

```
1  package v1ch05;
2
3  import module java.base;
4
5  /**
6   * This program demonstrates the use of reflection for manipulating arrays.
7   */
8  class CopyOfDemo {
9      void main() {
10         int[] a = { 1, 2, 3 };
11         a = (int[]) goodCopyOf(a, 10);
12         IO.println(Arrays.toString(a));
13
14         String[] b = { "Tom", "Dick", "Harry" };
15         b = (String[]) goodCopyOf(b, 10);
16         IO.println(Arrays.toString(b));
17
18         IO.println("The following call will generate an exception.");
19         b = (String[]) badCopyOf(b, 10);
20     }
21
22     /**
23      * This method attempts to grow an array by allocating a new array and
24      * copying all elements.
25      * @param a the array to grow
26      * @param newLength the new length
27      * @return a larger array that contains all elements of a. However, the
28      *         returned array has type Object[], not the same type as a
29      */
30     Object[] badCopyOf(Object[] a, int newLength) { // not useful
```

```
31          var newArray = new Object[newLength];
32          System.arraycopy(a, 0, newArray, 0, Math.min(a.length, newLength));
33          return newArray;
34      }
35
36      /**
37       * This method grows an array by allocating a new array of the same type and
38       * copying all elements.
39       * @param a the array to grow. This can be an object array or a primitive
40       *        type array
41       * @return a larger array that contains all elements of a
42       */
43      Object goodCopyOf(Object a, int newLength) {
44          Class cl = a.getClass();
45          if (!cl.isArray()) return null;
46          Class componentType = cl.getComponentType();
47          int length = Array.getLength(a);
48          Object newArray = Array.newInstance(componentType, newLength);
49          System.arraycopy(a, 0, newArray, 0, Math.min(length, newLength));
50          return newArray;
51      }
52  }
```

`java.lang.Class` 1.1

- `boolean isArray()`
  returns `true` if this object represents an array type.
- `Class getComponentType()`
  `Class componentType()` 12
  return the `Class` describing the component type if this object represents an array type, or `null` otherwise.
- `Class arrayType()` 12
  returns the `Class` describing the array type whose component type is represented by this object.

`java.lang.reflect.Array` 1.1

- `static Object get(Object array, int index)`
- `static xxx getXxx(Object array, int index)`
  These methods return the value of the given array that is stored at the given index. Here, `xxx` is one of the primitive types `boolean`, `byte`, `char`, `double`, `float`, `int`, `long`, or `short`.
- `static void set(Object array, int index, Object newValue)`
- `static setXxx(Object array, int index, xxx newValue)`
  These methods store a new value into the given array at the given index. Here, `xxx` is one of the primitive types `boolean`, `byte`, `char`, `double`, `float`, `int`, `long`, or `short`.

- `static int getLength(Object array)`
  returns the length of the given array.
- `static Object newInstance(Class componentType, int length)`
- `static Object newInstance(Class componentType, int[] lengths)`
  return a new array of the given component type with the given dimensions.

### 5.10.7. Invoking Arbitrary Methods and Constructors

In C and C++, you can execute an arbitrary function through a function pointer. On the surface, Java does not have method pointers—that is, ways of giving the location of a method to another method, so that the second method can invoke it later. In fact, the designers of Java have said that method pointers are dangerous and error-prone, and that Java interfaces and lambda expressions (discussed in the next chapter) are a superior solution. However, the reflection mechanism allows you to call arbitrary methods.

Recall that you can inspect a field of an object with the `get` method of the `Field` class. Similarly, the `Method` class has an `invoke` method that lets you call the method that is wrapped in the current `Method` object. The signature for the `invoke` method is

```
Object invoke(Object obj, Object... args)
```

The first parameter is the implicit argument, and the remaining objects provide the explicit arguments.

For a static method, the first argument is ignored—you can set it to `null`.

For example, if `m1` represents the `getName` method of the `Employee` class, the following code shows how you can call it:

```
String n = (String) m1.invoke(harry);
```

If the return type is a primitive type, the `invoke` method will return the wrapper type instead. For example, suppose that `m2` represents the `getSalary` method of the `Employee` class. Then, the returned object is actually a `Double`, and you must cast it accordingly. Use automatic unboxing to turn it into a `double`:

```
double s = (Double) m2.invoke(harry);
```

How do you obtain a `Method` object? You can, of course, call `getDeclaredMethods` and search through the returned array of `Method` ob-

jects until you find the method you want. Or, you can call the `getMethod` method of the `Class` class. This is similar to the `getField` method that takes a string with the field name and returns a `Field` object. However, there may be several methods with the same name, so you need to be careful that you get the right one. For that reason, you must also supply the parameter types of the desired method. The signature of `getMethod` is

```
Method getMethod(String name, Class... parameterTypes)
```

For example, here is how you can get method pointers to the `getName` and `raiseSalary` methods of the `Employee` class:

```
Method m1 = Employee.class.getMethod("getName");
Method m2 = Employee.class.getMethod("raiseSalary", double.class);
```

Use a similar approach for invoking arbitrary constructors. Supply the constructor's parameter types to the `Class.getConstructor` method, and supply the argument values to the `Constructor.newInstance` method:

```
Class cl = BigInteger.class;
    // or any other class with a constructor that has a String parameter
Constructor cons = cl.getConstructor(String.class);
Object obj = cons.newInstance("11235813213455");
```

---



**Note:** The `Method` and `Constructor` classes extend the `Executable` class. As of Java 17, the `Executable` class is sealed, permitting only `Method` and `Constructor` as subclasses.

---

Now that you have seen the rules for using `Method` objects, let's put them to work. **Listing 5.20** is a program that prints a table of values for a mathematical function such as `Math.sqrt` or `Math.sin`. The printout looks like this:

```
public static native double java.lang.Math.sqrt(double)
      1.0000 |      1.0000
      2.0000 |      1.4142
      3.0000 |      1.7321
      4.0000 |      2.0000
      5.0000 |      2.2361
      6.0000 |      2.4495
      7.0000 |      2.6458
      8.0000 |      2.8284
      9.0000 |      3.0000
     10.0000 |      3.1623
```

The code for printing a table is, of course, independent of the actual function that is being tabulated.

```
double dx = (to - from) / (n - 1);
for (double x = from; x <= to; x += dx) {
    double y = (Double) f.invoke(null, x);
    IO.println("%10.4f | %10.4f%n".formatted(x, y));
}
```

Here, `f` is an object of type `Method`. The first argument of `invoke` is `null` because we are calling a static method.

To tabulate the `Math.sqrt` function, we set `f` to

```
Math.class.getMethod("sqrt", double.class)
```

That is the method of the `Math` class that has the name `sqrt` and a single parameter of type `double`.

Listing 5.20 shows the complete code of the generic tabulator and a couple of test runs.

**Listing 5.20 v1ch05/MethodTableDemo.java**

```
1  package v1ch05;
2
3  import module java.base;
4
5  /**
6   * This program shows how to invoke methods through reflection.
7   */
8  class MethodTableDemo {
9      void main() throws Exception {
10         // get method pointers to the square and sqrt methods
11         Method square = this.getClass().getDeclaredMethod("square",
12                 double.class);
13         Method sqrt = Math.class.getMethod("sqrt", double.class);
14
15         // print tables of x- and y-values
16         printTable(1, 10, 10, square);
17         printTable(1, 10, 10, sqrt);
18     }
19
20     /**
21      * Returns the square of a number
22      * @param x a number
23      * @return x squared
24      */
25     static double square(double x) {
```

```
26          return x * x;
27      }
28
29      /**
30       * Prints a table with x- and y-values for a method
31       * @param from the lower bound for the x-values
32       * @param to the upper bound for the x-values
33       * @param n the number of rows in the table
34       * @param f a method with a double parameter and double return value
35       */
36      void printTable(double from, double to, int n, Method f) throws Exception {
37          // print out the method as table header
38          IO.println(f);
39
40          double dx = (to - from) / (n - 1);
41
42          for (double x = from; x <= to; x += dx) {
43              double y = (Double) f.invoke(null, x);
44              IO.println("%10.4f | %10.4f".formatted(x, y));
45          }
46      }
47  }
```

As this example clearly shows, you can do anything with `Method` objects that you can do with function pointers in C or delegates in C#. Just as in C, this style of programming is usually quite inconvenient, and always error-prone. What happens if you invoke a method with the wrong arguments? The `invoke` method throws an exception.

Also, the parameters and return values of `invoke` are necessarily of type `Object`. That means you must cast back and forth a lot. As a result, the compiler is deprived of the chance to check your code, so errors surface only during testing, when they are more tedious to find and fix. Moreover, code that uses reflection to get at method pointers is significantly slower than code that simply calls methods directly.

For that reason, I suggest that you use `Method` objects in your own programs only when absolutely necessary. Using interfaces and lambda expressions (the subject of the next chapter) is almost always a better idea.

`java.lang.reflect.Method` 1.1

- `public Object invoke(Object obj, Object[] args)`
  invokes the method described by this object, passing the given arguments and returning the value that the method returns. For static methods, pass `null` as the implicit argument. Pass primitive type values by using wrappers. Primitive type return values must be unwrapped.

## 5.11. Design Hints for Inheritance

I want to end this chapter with some hints that I have found useful when using inheritance.

1. *Place common operations and fields in the superclass.*
   This is why we put the name field into the `Person` class instead of replicating it in the `Employee` and `Student` classes.
2. *Don't use protected fields.*
   Some programmers think it is a good idea to define most instance fields as `protected`, "just in case," so that subclasses can access these fields if they need to. However, the `protected` mechanism doesn't give much protection, for two reasons. First, the set of subclasses is unbounded—anyone can form a subclass of your classes and then write code that directly accesses `protected` instance fields, thereby breaking encapsulation. And second, in Java, all classes in the same package have access to `protected` fields, whether or not they are subclasses.
   However, `protected` methods can be useful to indicate methods that are not ready for general use and should be redefined in subclasses.
3. *Use inheritance to model the "is–a" relationship.*
   Inheritance is a handy code-saver, but sometimes people overuse it. For example, suppose we need a `Contractor` class. Contractors have names and hire dates, but they do not have salaries. Instead, they are paid by the hour, and they do not stay around long enough to get a raise. There is the temptation to form a subclass `Contractor` from `Employee` and add an `hourlyWage` field.

   ```
   public class Contractor extends Employee {
       private double hourlyWage;
       . . .
   }
   ```

   This is *not* a good idea, however, because now each contractor object has both a salary and hourly wage field. It will cause you no end of grief when you implement methods for printing paychecks or tax forms. You will end up writing more code than you would have written by not inheriting in the first place.
   The contractor-employee relationship fails the "is–a" test. A contractor is not a special case of an employee.
4. *Don't use inheritance unless all inherited methods make sense.*
   Suppose we want to write a `Holiday` class. Surely every holiday is a day, and days can be expressed as instances of the `GregorianCalendar` class, so we can use inheritance.

   ```
   class Holiday extends GregorianCalendar { . . . }
   ```

Unfortunately, the set of holidays is not *closed* under the inherited operations. One of the public methods of `GregorianCalendar` is `add`. And `add` can turn holidays into nonholidays:

```
Holiday christmas;
christmas.add(Calendar.DAY_OF_MONTH, 12);
```

Therefore, inheritance is not appropriate in this example.
Note that this problem does not arise if you extend an immutable class. Suppose you have an immutable date class, similar to `LocalDate` but not `final`. If you form a `Holiday` subclass, there is no method that can turn a holiday into a nonholiday.

5. *Don't change the expected behavior when you override a method.*
   The substitution principle applies not just to syntax but, more importantly, to behavior. When you override a method, you should not unreasonably change its behavior. The compiler can't help you—it cannot check whether your redefinitions make sense. For example, you can "fix" the issue of the `add` method in the `Holiday` class by redefining `add`, perhaps to do nothing, or to throw an exception, or to move on to the next holiday.
   However, such a fix violates the substitution principle. The sequence of statements

```
int d1 = x.get(Calendar.DAY_OF_MONTH);
x.add(Calendar.DAY_OF_MONTH, 1);
int d2 = x.get(Calendar.DAY_OF_MONTH);
IO.println(d2 - d1);
```

   should have the *expected behavior*, no matter whether `x` is of type `GregorianCalendar` or `Holiday`.
   Of course, therein lies the rub. Reasonable and unreasonable people can argue at length about what the expected behavior is. For example, some authors argue that the substitution principle requires `Manager.equals` to ignore the `bonus` field because `Employee.equals` ignores it. These discussions are pointless if they occur in a vacuum. Ultimately, what matters is that you do not circumvent the intent of the original design when you override methods in subclasses.

6. *Use polymorphism, not type information.*
   Whenever you find code of the form

```
if (x is of type 1)
    action1(x);
else if (x is of type 2)
    action2(x);
```

think polymorphism.
Do $action_1$ and $action_2$ represent a common concept? If so, make the concept a method of a common superclass or interface of both types. Then, you can simply call

```
x.action();
```

and have the dynamic dispatch mechanism inherent in polymorphism launch the correct action.
Code that uses polymorphic methods or interface implementations is much easier to maintain and extend than code using multiple type tests.

7. *Don't overuse reflection.*
   The reflection mechanism lets you write programs with amazing generality, by detecting fields and methods at runtime. This capability can be extremely useful for systems programming, but it is usually not appropriate in applications. Reflection is fragile—with it, the compiler cannot help you find programming errors. Any errors are found at runtime and result in exceptions.

You have now seen how Java supports the fundamentals of object-oriented programming: classes, inheritance, and polymorphism. In the next chapter, I will tackle two advanced topics that are very important for using Java effectively: interfaces and lambda expressions.

Previous chapter
< 4. Objects and Classes

Next chapter
6. Interfaces, Lambda Expressions, and Inner Classes >